

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 4**



Sorting

Disusun oleh :
Vivaldi Fachmy Fahlevi **NIM.2510817210017**

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 4

Laporan Praktikum Basis Data I Modul 4 : Sorting ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Vivaldi Fachmy Fahlevi
NIM : 2510817210017

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir.Muhmmad Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR.....	5
DAFTAR TABEL	6
SORTING ALGORITHM	1
A. Source Code.....	1
B. Output Program	13
C. Pembahasan	16
SOAL 1.....	34
A. Bubble Sort.....	35
B. Selection Sort.....	37
C. Insertion Sort	39
D. Merge Sort	41
E. Quick Sort	43
F. Radix Sort	45
SOAL 2.....	47
A. Bubble Sort.....	48
B. Selection Sort.....	50
C. Insertion Sort	52
D. Merge Sort	54
E. Quick Sort	56
F. Radix Sort.....	59
SOAL 3.....	63
A. Bubble Sort.....	64

B. Selection Sort.....	66
C. Insertion Sort	68
D. Merge Sort	70
E. Quick Sort	72
F. Radix Sort.....	74
SOAL 4	76
TAUTAN GIT	79

DAFTAR GAMBAR

Gambar 1 Hasil Pengujian Bubble Sort.....	13
Gambar 2 Hasil Pengujian Selection Sort	13
Gambar 3 Hasil Pengujian Insertion Sort.....	14
Gambar 4 Hasil Pengujian Merge Sort.....	14
Gambar 5 Hasil Pengujian Quick Sort	14
Gambar 6 Hasil Pengujian Radix Sort.....	15
Gambar 7 Hasil Pengujian Bubble Sort	35
Gambar 8 Hasil Pengujian Selection Sort.....	37
Gambar 9 Hasil Pengujian Insertion Sort.....	39
Gambar 10 Hasil Pengujian Merge Sort	41
Gambar 11 Hasil Pengujian Quick Sort	43
Gambar 12 Hasil Pengujian Radix Sort	45

DAFTAR TABEL

Tabel 1 Source Code file algorithm.cpp	1
Tabel 2 Source Code File sorting_algorithms.h.....	7
Tabel 3 Source Code File metric.h.....	7
Tabel 4 Source Code File report.h	8
Tabel 5 Source Code File reporter.cpp	8
Tabel 6 Source Code File main.cpp	9
Tabel 7 Source Code File data_generator.h	11
Tabel 8 Source Code File generator.cpp	11

SORTING ALGORITHM

A. Source Code

Tabel 1 Source Code file algorithm.cpp

1	#include "sorting_algorithms.h"
2	#include <algorithm>
3	#include <chrono>
4	
5	using Clock = std::chrono::high_resolution_clock;
6	
7	// BUBBLE SORT
8	void bubble_sort(std::vector<int>& data, Metrics&
9	m) {
10	int n = data.size();
11	auto start = Clock::now();
12	for (int i = 0; i < n - 1; i++) {
13	for (int j = 0; j < n - i - 1; j++) {
14	
15	m.comparisons++;
16	
17	if (data[j] > data[j + 1]) {
18	std::swap(data[j], data[j + 1]);
19	m.swaps++;
20	}
21	}
22	}
23	
24	m.time_ms =
25	std::chrono::duration<double,
26	std::milli>(
27	Clock::now() - start).count();
28	}
29	// SELECTION SORT
30	void selection_sort(std::vector<int>& data,
31	Metrics& m) {
32	int n = data.size();
33	auto start = Clock::now();
34	for (int i = 0; i < n - 1; i++) {
35	
36	int minIndex = i;
37	
38	for (int j = i + 1; j < n; j++) {
39	
40	m.comparisons++;

```

41
42         if (data[j] < data[minIndex]) {
43             minIndex = j;
44         }
45     }
46
47     if (minIndex != i) {
48         std::swap(data[i], data[minIndex]);
49         m.swaps++;
50     }
51 }
52
53     m.time_ms =
54         std::chrono::duration<double,
55 std::chrono::milli>(
56         Clock::now() - start).count();
57 }
58 // INSERTION SORT
59 void insertion_sort(std::vector<int>& data,
60 Metrics& m) {
61     int n = data.size();
62     auto start = Clock::now();
63
64     for (int i = 1; i < n; i++) {
65
66         int key = data[i];
67         int j = i - 1;
68
69         while (j >= 0) {
70
71             m.comparisons++;
72
73             if (data[j] > key) {
74                 data[j + 1] = data[j];
75                 m.shifts++;
76                 j--;
77             }
78             else {
79                 break;
80             }
81
82             data[j + 1] = key;
83         }
84
85     m.time_ms =
86         std::chrono::duration<double,
87 std::chrono::milli>(

```


87	Clock::now() - start).count();
88	}
89	
90	// MERGE SORT HELPER
91	void merge(std::vector<int>& data,
92	int left,
93	int mid,
94	int right,
95	Metrics& m) {
96	
97	int n1 = mid - left + 1;
98	int n2 = right - mid;
99	
100	std::vector<int> L(n1);
101	std::vector<int> R(n2);
102	
103	for (int i = 0; i < n1; i++)
104	L[i] = data[left + i];
105	
106	for (int j = 0; j < n2; j++)
107	R[j] = data[mid + 1 + j];
108	
109	int i = 0;
110	int j = 0;
111	int k = left;
112	
113	while (i < n1 && j < n2) {
114	
115	m.comparisons++;
116	
117	if (L[i] <= R[j]) {
118	data[k] = L[i];
119	i++;
120	}
121	else {
122	data[k] = R[j];
123	j++;
124	}
125	
126	k++;
127	}
128	
129	while (i < n1) {
130	data[k] = L[i];
131	i++;
132	k++;
133	}
134	
135	while (j < n2) {

```

136         data[k] = R[j];
137         j++;
138         k++;
139     }
140 }
141
142 void merge_sort_recursive(std::vector<int>& data,
143                          int left,
144                          int right,
145                          Metrics& m) {
146
147     m.recursive_calls++;
148
149     if (left >= right)
150         return;
151
152     int mid = left + (right - left) / 2;
153
154     merge_sort_recursive(data, left, mid, m);
155     merge_sort_recursive(data, mid + 1, right,
156 m);
157
158     merge(data, left, mid, right, m);
159 }
160
161 // MERGE SORT
162 void merge_sort(std::vector<int>& data, Metrics&
163 m) {
164     auto start = Clock::now();
165
166     if (!data.empty()) {
167         merge_sort_recursive(data, 0, data.size()
168 - 1, m);
169     }
170
171     m.time_ms =
172         std::chrono::duration<double,
173 std::milli>(
174         Clock::now() - start).count();
175 }
176
177 // QUICK SORT HELPER
178 int partition(std::vector<int>& data,
179              int low,
180              int high,
181              Metrics& m) {
182
183     int pivot = data[high];
184     int i = low - 1;

```

```

181
182     for (int j = low; j < high; j++) {
183
184         m.comparisons++;
185
186         if (data[j] < pivot) {
187             i++;
188             std::swap(data[i], data[j]);
189             m.swaps++;
190         }
191     }
192
193     std::swap(data[i + 1], data[high]);
194     m.swaps++;
195
196     return i + 1;
197 }
198
199 void quick_sort_recursive(std::vector<int>& data,
200                          int low,
201                          int high,
202                          Metrics& m) {
203
204     m.recursive_calls++;
205
206     if (low < high) {
207
208         int pi = partition(data, low, high, m);
209
210         quick_sort_recursive(data, low, pi - 1,
211 m);
212         quick_sort_recursive(data, pi + 1, high,
213 m);
214     }
215 }
216
217 // QUICK SORT
218 void quick_sort(std::vector<int>& data, Metrics&
219 m) {
220     auto start = Clock::now();
221
222     if (!data.empty()) {
223         quick_sort_recursive(data, 0, data.size()
224 - 1, m);
225     }
226
227     m.time_ms =
228         std::chrono::duration<double,
229 std::milli>(

```

```

225         Clock::now() - start).count();
226     }
227
228     // RADIX SORT HELPER
229     void counting_sort(std::vector<int>& data,
230                       int exp,
231                       Metrics& m) {
232
233         int n = data.size();
234
235         std::vector<int> output(n);
236         int count[10] = {0};
237
238         for (int i = 0; i < n; i++) {
239             count[(data[i] / exp) % 10]++;
240             m.array_accesses++;
241         }
242
243         for (int i = 1; i < 10; i++) {
244             count[i] += count[i - 1];
245         }
246
247         for (int i = n - 1; i >= 0; i--) {
248
249             int index = (data[i] / exp) % 10;
250
251             output[count[index] - 1] = data[i];
252             count[index]--;
253
254             m.array_accesses++;
255         }
256
257         for (int i = 0; i < n; i++) {
258             data[i] = output[i];
259             m.array_accesses++;
260         }
261     }
262
263     // RADIX SORT
264     void radix_sort(std::vector<int>& data, Metrics&
265 m) {
266         if (data.empty()) return;
267
268         auto start = Clock::now();
269
270         int maxVal = *std::max_element(data.begin(),
269 data.end());

```

271	for (int exp = 1; maxVal / exp > 0; exp *=
272	10) {
273	counting_sort(data, exp, m);
274	}
275	m.time_ms =
276	std::chrono::duration<double,
277	std::milli>(
278	Clock::now() - start).count();
	}

Tabel 2 Source Code File sorting_algorithms.h

1	#pragma once
2	#include <vector>
3	#include "metrics.h"
4	
5	void bubble_sort (std::vector<int>& data,
	Metrics& m);
6	void selection_sort (std::vector<int>& data,
	Metrics& m);
7	void insertion_sort (std::vector<int>& data,
	Metrics& m);
8	void merge_sort (std::vector<int>& data,
	Metrics& m);
9	void radix_sort (std::vector<int>& data,
	Metrics& m);
10	void quick_sort (std::vector<int>& data,
	Metrics& m);

Tabel 3 Source Code File metric.h

1	#include "sorting_algorithms.h"
2	#include <algorithm>
3	#include <chrono>
4	
5	using Clock = std::chrono::high_resolution_clock;
6	
7	// BUBBLE SORT
8	void bubble_sort(std::vector<int>& data, Metrics&
9	m) {
10	int n = data.size();
11	auto start = Clock::now();
12	for (int i = 0; i < n - 1; i++) {
13	for (int j = 0; j < n - i - 1; j++) {
14	
15	m.comparisons++;

16	
17	if (data[j] > data[j + 1]) {
18	std::swap(data[j], data[j + 1]);
19	m.swaps++;
20	}
21	}
22	}
23	
24	m.time_ms =
	std::chrono::duration<double,
25	std::milli>(
26	Clock::now() - start).count();

Tabel 4 Source Code File report.h

1	#pragma once
2	#include <vector>
3	#include "metrics.h"
4	
5	void printMetrics(const Metrics& m);
6	
7	void printTableHeader();

Tabel 5 Source Code File reporter.cpp

1	#include "reporter.h"
2	#include <iostream>
3	#include <iomanip>
4	
5	static const int W_ALG = 16;
6	static const int W_INPUT = 16;
7	static const int W_N = 8;
8	static const int W_STAT = 14;
9	static const int W_TIME = 16;
10	
11	void printTableHeader() {
12	std::cout << std::left
	<< std::setw(W_ALG) <<
13	"Algorithm"
	<< std::setw(W_INPUT) << "Input
14	Type"
15	<< std::setw(W_N) << "N"
	<< std::setw(W_STAT) <<
16	"Comparisons"
17	<< std::setw(W_STAT) << "Swaps"
18	<< std::setw(W_STAT) << "Shifts"
	<< std::setw(W_STAT) <<
19	"Rec.Calls"

```

20         << std::setw(W_STAT) <<
    "Arr.Access"
21         << std::setw(W_TIME) << "Time(ms)"
22         << "\n";
    std::cout << std::string(W_ALG + W_INPUT +
23 W_N + W_STAT*5 + W_TIME, '-') << "\n";
24 }
25
26 void printMetrics(const Metrics& m) {
27     std::cout << std::left
    << std::setw(W_ALG) <<
28 m.algorithm_name
    << std::setw(W_INPUT) <<
29 m.input_type
30     << std::setw(W_N) << m.n
    << std::setw(W_STAT) <<
31 m.comparisons
32     << std::setw(W_STAT) << m.swaps
33     << std::setw(W_STAT) << m.shifts
    << std::setw(W_STAT) <<
34 m.recursive_calls
    << std::setw(W_STAT) <<
35 m.array_accesses
    << std::fixed <<
36 std::setprecision(6)
37     << std::setw(W_TIME) << m.time_ms
38     << "\n";
39 }

```

Tabel 6 Source Code File main.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <functional>
4  #include <string>
5
6  #include "metrics.h"
7  #include "data_generator.h"
8  #include "sorting_algorithms.h"
9  #include "reporter.h"
10
11 static const std::vector<int> N_VALUES = { 100,
12 1'000, 10'000 };
13
14 static const std::vector<InputType> INPUT_TYPES =
15 {
16     InputType::RANDOM,
17     InputType::SORTED,
18     InputType::REVERSE_SORTED,

```

```

17     InputType::NEARLY_SORTED,
18     InputType::DUPLICATES,
19 };
20
21 static const unsigned int SEED = 42;
22 using SortFn =
23     std::function<void(std::vector<int>&, Metrics&)>;
24
25 static const std::vector<std::pair<std::string,
26     SortFn>> ALGORITHMS = {
27     { "Bubble Sort",    bubble_sort    },
28     { "Selection Sort", selection_sort },
29     { "Insertion Sort", insertion_sort },
30     { "Merge Sort",     merge_sort     },
31     { "Quick Sort",     quick_sort     },
32     { "Radix Sort",     radix_sort     },
33 };
34
35 int main() {
36     std::cout <<
37     "=====
38     =\n";
39     std::cout << "          SORTING ALGORITHM -
40     ANALYSIS\n";
41     std::cout <<
42     "=====
43     =\n";
44
45     for (const auto& [name, sortFn] : ALGORITHMS)
46     {
47         std::cout << "\n>> " << name << "\n";
48         printTableHeader();
49
50         for (int n : N_VALUES) {
51             for (const auto& inputType :
52                 INPUT_TYPES) {
53                 std::vector<int> data =
54                 generateData(n, inputType, SEED);
55
56                 Metrics m;
57                 m.algorithm_name = name;
58                 m.input_type     =
59                 inputTypeName(inputType);
60                 m.n               = n;
61
62                 sortFn(data, m);
63                 printMetrics(m);
64             }
65         }
66     }
67 }

```


55	}
56	}
57	
58	std::cout << "\nDone.\n";
59	return 0;
60	}

Tabel 7 Source Code File data_generator.h

1	#pragma once
2	#include <vector>
3	#include <string>
4	
5	enum class InputType {
6	RANDOM,
7	SORTED,
8	REVERSE_SORTED,
9	NEARLY_SORTED,
10	DUPLICATES
11	};
12	
13	std::string inputTypeName(InputType type);
14	
15	std::vector<int> generateData(int n, InputType type, unsigned int seed = 42);

Tabel 8 Source Code File generator.cpp

1	#include "data_generator.h"
2	#include <algorithm>
3	#include <numeric>
4	#include <random>
5	#include <stdexcept>
6	
7	std::string inputTypeName(InputType type) {
8	switch (type) {
9	case InputType::RANDOM: return
10	"Random";
11	case InputType::SORTED: return
12	"Sorted";
13	case InputType::REVERSE_SORTED: return
14	"Reverse Sorted";
15	case InputType::NEARLY_SORTED: return
16	"Nearly Sorted";
17	case InputType::DUPLICATES: return
18	"Many Duplicates";
19	default: return
20	"Unknown";
21	}

```

16 }
17
18 std::vector<int> generateData(int n, InputType
19 type, unsigned int seed) {
20     if (n <= 0) throw std::invalid_argument("N
21 harus > 0");
22
23     std::mt19937 rng(seed);
24     std::vector<int> data(n);
25
26     switch (type) {
27         case InputType::RANDOM: {
28             std::uniform_int_distribution<int>
29             dist(1, n * 10);
30             for (auto& x : data) x = dist(rng);
31             break;
32         }
33         case InputType::SORTED: {
34             std::iota(data.begin(), data.end(),
35             1);
36             break;
37         }
38         case InputType::REVERSE_SORTED: {
39             std::iota(data.begin(), data.end(),
40             1);
41             std::reverse(data.begin(),
42             data.end());
43             break;
44         }
45         case InputType::NEARLY_SORTED: {
46             std::iota(data.begin(), data.end(),
47             1);
48             int swaps = std::max(1, n / 20);
49             std::uniform_int_distribution<int>
50             idx(0, n - 1);
51             for (int i = 0; i < swaps; i++) {
52                 std::swap(data[idx(rng)],
53                 data[idx(rng)]);
54             }
55             break;
56         }
57         case InputType::DUPLICATES: {
58             int range = std::max(2, n / 10);

```

```

55         std::uniform_int_distribution<int>
56         dist(1, range);
57         for (auto& x : data) x = dist(rng);
58         break;
59     }
60 }
61 return data;
62 }

```

B. Output Program

>> Bubble Sort

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Bubble Sort	Random	100	4950	2564	0	0	0	0.025500
Bubble Sort	Sorted	100	4950	0	0	0	0	0.003100
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.037600
Bubble Sort	Nearly Sorted	100	4950	296	0	0	0	0.005100
Bubble Sort	Many Duplicates	100	4950	2315	0	0	0	0.015100
Bubble Sort	Random	1000	499500	243190	0	0	0	1.408100
Bubble Sort	Sorted	1000	499500	0	0	0	0	0.441800
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	2.724500
Bubble Sort	Nearly Sorted	1000	499500	25778	0	0	0	0.586900
Bubble Sort	Many Duplicates	1000	499500	240808	0	0	0	1.424000
Bubble Sort	Random	10000	49995000	25118148	0	0	0	149.790900
Bubble Sort	Sorted	10000	49995000	0	0	0	0	33.248400
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	266.830100
Bubble Sort	Nearly Sorted	10000	49995000	3119352	0	0	0	43.251300
Bubble Sort	Many Duplicates	10000	49995000	25093086	0	0	0	150.826500

Gambar 1 Hasil Pengujian Bubble Sort

>> Selection Sort

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	95	0	0	0	0.007700
Selection Sort	Sorted	100	4950	0	0	0	0	0.006100
Selection Sort	Reverse Sorted	100	4950	50	0	0	0	0.009900
Selection Sort	Nearly Sorted	100	4950	4	0	0	0	0.006300
Selection Sort	Many Duplicates	100	4950	89	0	0	0	0.007700
Selection Sort	Random	1000	499500	995	0	0	0	0.782600
Selection Sort	Sorted	1000	499500	0	0	0	0	1.105500
Selection Sort	Reverse Sorted	1000	499500	500	0	0	0	0.781900
Selection Sort	Nearly Sorted	1000	499500	50	0	0	0	0.776500
Selection Sort	Many Duplicates	1000	499500	983	0	0	0	0.949100
Selection Sort	Random	10000	49995000	9992	0	0	0	85.182500
Selection Sort	Sorted	10000	49995000	0	0	0	0	84.525900
Selection Sort	Reverse Sorted	10000	49995000	5000	0	0	0	85.146000
Selection Sort	Nearly Sorted	10000	49995000	500	0	0	0	84.712600
Selection Sort	Many Duplicates	10000	49995000	9981	0	0	0	84.728800

Gambar 2 Hasil Pengujian Selection Sort

>> Insertion Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Insertion Sort	Random	100	2656	0	2564	0	0	0.002900
Insertion Sort	Sorted	100	99	0	0	0	0	0.000200
Insertion Sort	Reverse Sorted	100	4950	0	4950	0	0	0.002600
Insertion Sort	Nearly Sorted	100	395	0	296	0	0	0.000500
Insertion Sort	Many Duplicates	100	2412	0	2315	0	0	0.001800
Insertion Sort	Random	1000	244181	0	243190	0	0	0.106400
Insertion Sort	Sorted	1000	999	0	0	0	0	0.001300
Insertion Sort	Reverse Sorted	1000	499500	0	499500	0	0	0.228600
Insertion Sort	Nearly Sorted	1000	26775	0	25778	0	0	0.020600
Insertion Sort	Many Duplicates	1000	241801	0	240808	0	0	0.116000
Insertion Sort	Random	10000	25128135	0	25118148	0	0	10.711900
Insertion Sort	Sorted	10000	9999	0	0	0	0	0.012500
Insertion Sort	Reverse Sorted	10000	49995000	0	49995000	0	0	24.324700
Insertion Sort	Nearly Sorted	10000	3129351	0	3119352	0	0	1.583400
Insertion Sort	Many Duplicates	10000	25103078	0	25093086	0	0	12.004100

Gambar 3 Hasil Pengujian Insertion Sort

>> Merge Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Merge Sort	Random	100	535	0	0	199	0	0.013100
Merge Sort	Sorted	100	356	0	0	199	0	0.008700
Merge Sort	Reverse Sorted	100	316	0	0	199	0	0.008500
Merge Sort	Nearly Sorted	100	453	0	0	199	0	0.008400
Merge Sort	Many Duplicates	100	536	0	0	199	0	0.012800
Merge Sort	Random	1000	8690	0	0	1999	0	0.099400
Merge Sort	Sorted	1000	5044	0	0	1999	0	0.062800
Merge Sort	Reverse Sorted	1000	4932	0	0	1999	0	0.062300
Merge Sort	Nearly Sorted	1000	7628	0	0	1999	0	0.085700
Merge Sort	Many Duplicates	1000	8679	0	0	1999	0	0.093900
Merge Sort	Random	10000	120465	0	0	19999	0	1.015000
Merge Sort	Sorted	10000	69008	0	0	19999	0	0.769000
Merge Sort	Reverse Sorted	10000	64608	0	0	19999	0	0.943900
Merge Sort	Nearly Sorted	10000	111524	0	0	19999	0	0.712700
Merge Sort	Many Duplicates	10000	120461	0	0	19999	0	1.364500

Gambar 4 Hasil Pengujian Merge Sort

>> Quick Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	678	394	0	129	0	0.002300
Quick Sort	Sorted	100	4950	5049	0	199	0	0.003500
Quick Sort	Reverse Sorted	100	4950	2549	0	199	0	0.003500
Quick Sort	Nearly Sorted	100	2638	2607	0	191	0	0.003900
Quick Sort	Many Duplicates	100	909	236	0	181	0	0.002000
Quick Sort	Random	1000	10537	6125	0	1341	0	0.027700
Quick Sort	Sorted	1000	499500	500499	0	1999	0	0.349300
Quick Sort	Reverse Sorted	1000	499500	250499	0	1999	0	0.282900
Quick Sort	Nearly Sorted	1000	35189	25081	0	1611	0	0.041900
Quick Sort	Many Duplicates	1000	12685	4920	0	1801	0	0.036600
Quick Sort	Random	10000	147606	77996	0	13315	0	0.382400
Quick Sort	Sorted	10000	49995000	50004999	0	19999	0	32.773400
Quick Sort	Reverse Sorted	10000	49995000	25004999	0	19999	0	27.840300
Quick Sort	Nearly Sorted	10000	622473	458547	0	16511	0	0.693600
Quick Sort	Many Duplicates	10000	181016	67083	0	18003	0	0.302600

Gambar 5 Hasil Pengujian Quick Sort

```
>> Radix Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Radix Sort	Random	100	0	0	0	0	900	0.002700
Radix Sort	Sorted	100	0	0	0	0	900	0.001800
Radix Sort	Reverse Sorted	100	0	0	0	0	900	0.002200
Radix Sort	Nearly Sorted	100	0	0	0	0	900	0.001800
Radix Sort	Many Duplicates	100	0	0	0	0	600	0.001400
Radix Sort	Random	1000	0	0	0	0	12000	0.015000
Radix Sort	Sorted	1000	0	0	0	0	12000	0.013200
Radix Sort	Reverse Sorted	1000	0	0	0	0	12000	0.016200
Radix Sort	Nearly Sorted	1000	0	0	0	0	12000	0.013100
Radix Sort	Many Duplicates	1000	0	0	0	0	9000	0.015300
Radix Sort	Random	10000	0	0	0	0	150000	0.177400
Radix Sort	Sorted	10000	0	0	0	0	150000	0.151800
Radix Sort	Reverse Sorted	10000	0	0	0	0	150000	0.151500
Radix Sort	Nearly Sorted	10000	0	0	0	0	150000	0.158400
Radix Sort	Many Duplicates	10000	0	0	0	0	120000	0.125600

Gambar 6 Hasil Pengujian Radix Sort

C. Pembahasan

Algoritma **Bubble Sort** bekerja dengan cara membandingkan dua data yang berdekatan kemudian menukarnya apabila urutannya salah. Proses ini dilakukan berulang kali sampai seluruh data tersusun dengan benar. Pada setiap putaran, nilai terbesar akan bergerak ke bagian akhir sehingga prosesnya mirip seperti gelembung yang naik ke permukaan.

Pada kode `sorting_algorithms.cpp`, proses Bubble Sort dimulai pada baris [8] melalui fungsi `bubble_sort`. Jumlah data disimpan ke variabel `n` pada baris [9], kemudian waktu mulai dihitung pada baris [10]. Perulangan utama pada baris [12] digunakan untuk menentukan jumlah putaran sorting, sedangkan perulangan di baris [13] digunakan untuk membandingkan elemen yang berdekatan. Setiap kali terjadi pengecekan, nilai `comparisons` ditambah pada baris [15]. Kondisi utama Bubble Sort terdapat pada baris [17], yaitu ketika elemen kiri lebih besar daripada elemen kanan maka kedua data ditukar menggunakan `std::swap` pada baris [18]. Setelah pertukaran terjadi, jumlah swap dicatat pada baris [19]. Setelah seluruh proses selesai, waktu eksekusi disimpan pada baris [23–24].

Selection Sort bekerja dengan cara mencari nilai terkecil dari bagian data yang belum terurut, kemudian menempatkannya ke posisi yang benar. Algoritma ini terus mengulang proses pencarian nilai minimum hingga seluruh data tersusun rapi.

Pada kode `sorting_algorithms.cpp`, fungsi `selection_sort` dimulai pada baris [28]. Variabel `minIndex` pada baris [35] digunakan untuk menyimpan posisi nilai terkecil sementara. Perulangan di baris [37] digunakan untuk mencari nilai paling kecil pada bagian data yang belum terurut. Setiap perbandingan dicatat pada baris [39]. Jika ditemukan nilai yang lebih kecil, posisi minimum diperbarui pada baris [41–42]. Setelah pencarian selesai, data ditukar pada baris [46] apabila posisi minimum berubah. Jumlah pertukaran kemudian dicatat pada baris [47]. Waktu eksekusi dihitung pada bagian akhir fungsi di baris [51–52].

Insertion Sort bekerja dengan cara mengambil satu data lalu menyisipkannya ke posisi yang sesuai pada bagian data yang sudah terurut. Cara kerjanya mirip seperti menyusun kartu satu per satu ke posisi yang benar.

Pada kode, **sorting_algorithms.cpp** fungsi `insertion_sort` dimulai pada baris [56]. Perulangan utama pada baris [60] mengambil data satu per satu mulai dari indeks kedua. Nilai yang akan dipindahkan disimpan ke variabel `key` pada baris [62]. Selanjutnya perulangan `while` pada baris [65] digunakan untuk menggeser data yang lebih besar dari `key`. Setiap pengecekan dicatat pada baris [67]. Jika data sebelumnya lebih besar, maka data digeser ke kanan pada baris [70], jumlah `shift` ditambah pada baris [71], lalu indeks mundur pada baris [72]. Setelah posisi yang tepat ditemukan, nilai `key` dimasukkan kembali pada baris [79]. Waktu eksekusi disimpan pada baris [83–84].

Merge Sort bekerja dengan cara membagi data menjadi bagian-bagian kecil, kemudian menggabungkannya kembali dalam keadaan terurut. Algoritma ini menggunakan konsep pemecahan data secara bertahap lalu penyatuan kembali.

Pada kode **sorting_algorithms.cpp**, proses penggabungan dilakukan dalam fungsi `merge` yang dimulai pada baris [88]. Data dibagi menjadi dua bagian menggunakan array sementara `L` dan `R` pada baris [95–96]. Perbandingan antar kedua bagian dilakukan pada baris [111], lalu elemen yang lebih kecil dimasukkan kembali ke array utama pada baris [113–120]. Fungsi rekursif utama berada pada `merge_sort_recursive` di baris [140]. Jumlah pemanggilan rekursif dicatat pada baris [145]. Data dibagi menjadi dua bagian menggunakan nilai tengah pada baris [150], lalu fungsi memanggil dirinya sendiri pada baris [152–153]. Setelah kedua bagian selesai diproses, data digabung kembali melalui fungsi `merge` pada baris [155]. Fungsi utama `merge_sort` dimulai pada baris [159] dan menjalankan proses rekursif pada baris [163].

Quick Sort bekerja dengan memilih satu nilai sebagai titik acuan atau `pivot`, kemudian memisahkan data yang lebih kecil dan lebih besar dari `pivot`. Setelah itu proses dilakukan kembali pada bagian kiri dan kanan sampai seluruh data terurut.

Pada kode **sorting_algorithms.cpp**, proses pembagian data dilakukan pada fungsi `partition` di baris [173]. Pivot dipilih dari elemen terakhir pada baris [177]. Perulangan pada baris [180] digunakan untuk membandingkan setiap data dengan pivot. Jika data lebih kecil dari pivot, maka posisi indeks dinaikkan pada baris [185] dan data ditukar pada baris [186]. Setelah seluruh data diperiksa, pivot dipindahkan ke posisi yang benar pada baris [191]. Fungsi rekursif `quick_sort_recursive` dimulai pada baris [198]. Jumlah pemanggilan rekursif dicatat pada baris [203]. Setelah posisi pivot ditemukan, bagian kiri dan kanan diproses kembali secara terpisah pada baris [209–210]. Fungsi utama `quick_sort` dimulai pada baris [216] dan menjalankan rekursi pada baris [220].

Radix Sort bekerja dengan mengurutkan data berdasarkan digit angka, dimulai dari digit paling belakang menuju digit paling depan. Algoritma ini tidak membandingkan angka secara langsung seperti algoritma sorting lainnya.

Pada kode **sorting_algorithms.cpp**, proses pengurutan per digit dilakukan pada fungsi `counting_sort` di baris [230]. Array `count` pada baris [236] digunakan untuk menghitung jumlah kemunculan digit tertentu. Perulangan pada baris [238] membaca digit dari setiap angka lalu mencatat jumlah akses array pada baris [240]. Setelah itu dilakukan perhitungan posisi digit pada baris [243–245]. Data kemudian dipindahkan ke array sementara `output` pada baris [251], lalu jumlah akses array ditambah pada baris [254]. Setelah proses selesai, data dikembalikan ke array utama pada baris [258–261]. Fungsi utama `radix_sort` dimulai pada baris [266]. Nilai terbesar dicari pada baris [271] untuk menentukan jumlah digit maksimum. Perulangan pada baris [273] menjalankan `counting_sort` untuk setiap digit sampai seluruh data terurut.

Pada file header **algoritma_sorting.h** terdapat deklarasi beberapa algoritma sorting yang nantinya akan digunakan pada program. File ini berfungsi sebagai penghubung agar fungsi sorting dapat dipanggil dari file lain tanpa perlu menuliskan ulang isi fungsi. Pada baris [1] digunakan `#pragma once` yang berfungsi agar file header hanya dibaca satu kali oleh compiler sehingga tidak terjadi duplikasi deklarasi. Kemudian pada baris [2] program memanggil library vector karena seluruh algoritma menggunakan `std::vector<int>` sebagai tempat penyimpanan data angka yang akan diurutkan. Pada baris [3] dipanggil file `metrics.h` yang digunakan untuk mencatat informasi pengujian seperti jumlah comparison, swap, shift, recursion call, array access, dan waktu eksekusi.

Algoritma pertama adalah Bubble Sort yang dideklarasikan pada baris [5]. Algoritma ini bekerja dengan cara membandingkan dua elemen yang berdekatan lalu menukarnya jika urutannya salah. Proses tersebut dilakukan berulang dari awal hingga akhir data sampai seluruh elemen tersusun dengan benar. Semakin besar data yang belum terurut, semakin banyak proses pertukaran yang dilakukan.

Selection Sort pada baris [6] bekerja dengan cara mencari nilai terkecil dari bagian data yang belum terurut, kemudian menempatkannya ke posisi yang benar. Proses ini dilakukan sedikit demi sedikit dari kiri ke kanan hingga seluruh data tersusun rapi. Algoritma ini memiliki jumlah swap yang relatif sedikit karena pertukaran hanya dilakukan setelah nilai minimum ditemukan.

Insertion Sort pada baris [7] bekerja seperti proses menyusun kartu secara manual. Algoritma mengambil satu elemen lalu menyisipkannya ke posisi yang benar pada bagian data yang sudah terurut. Jika data sudah hampir rapi, proses sorting dapat berjalan sangat cepat karena hanya membutuhkan sedikit perpindahan data.

Merge Sort pada baris [8] menggunakan metode membagi data menjadi bagian-bagian kecil terlebih dahulu, kemudian menggabungkannya kembali dalam keadaan terurut. Proses pembagian dilakukan terus menerus sampai data menjadi sangat kecil, lalu setiap bagian digabung kembali secara bertahap hingga seluruh data tersusun rapi. Algoritma ini dikenal stabil dan cukup cepat untuk data berukuran besar.

Radix Sort pada baris [9] bekerja dengan cara mengurutkan angka berdasarkan digitnya mulai dari digit paling belakang menuju digit paling depan. Algoritma ini tidak membandingkan angka secara langsung, melainkan mengelompokkan data berdasarkan nilai digit pada setiap tahap. Karena itu Radix Sort dapat bekerja sangat cepat pada data angka dengan jumlah digit yang tidak terlalu besar.

Quick Sort pada baris [10] bekerja dengan memilih satu elemen sebagai pivot atau titik acuan. Data kemudian dibagi menjadi dua bagian, yaitu elemen yang lebih kecil dari pivot dan elemen yang lebih besar dari pivot. Setelah itu proses yang sama dilakukan kembali pada masing-masing bagian sampai seluruh data terurut. Quick Sort terkenal sangat cepat pada kondisi normal, tetapi performanya dapat menurun jika pemilihan pivot kurang tepat.

Pada file header **metrics.h** dibuat sebuah struct bernama Metrics pada baris [5] yang berfungsi untuk menyimpan seluruh hasil pengukuran performa algoritma sorting. Struct ini digunakan agar setiap algoritma dapat mencatat informasi pengujian seperti jumlah comparison, swap, waktu eksekusi, dan operasi lainnya dalam satu tempat yang terorganisir.

Pada baris [1] digunakan `#pragma once` agar file header hanya diproses satu kali oleh compiler sehingga tidak terjadi duplikasi. Kemudian pada baris [2] program memanggil library string karena beberapa data seperti nama algoritma dan jenis input menggunakan tipe data string. Pada baris [3] digunakan library chrono yang nantinya dipakai untuk menghitung waktu eksekusi algoritma sorting.

Bagian utama dimulai pada baris [5] dengan pembuatan struct Metrics. Struct ini berfungsi seperti wadah yang menyimpan seluruh informasi hasil pengujian algoritma. Pada baris [6] dan [7], terdapat variabel `algorithm_name` dan `input_type` yang digunakan untuk menyimpan nama algoritma serta jenis data yang diuji, misalnya random, sorted, atau reverse sorted. Kemudian pada baris [8], variabel `n` digunakan untuk menyimpan jumlah data yang diproses.

Pada baris [10], variabel `comparisons` digunakan untuk menghitung berapa kali algoritma melakukan perbandingan antar elemen data. Nilai ini penting karena semakin banyak comparison biasanya waktu proses juga semakin besar. Pada baris [11], `swaps` digunakan untuk menghitung jumlah pertukaran posisi data selama proses sorting berlangsung. Selanjutnya pada baris [12], `shifts` digunakan khusus untuk algoritma seperti Insertion Sort yang memindahkan elemen dengan cara menggeser data, bukan menukar langsung.

Baris [13] berisi `recursive_calls` yang digunakan untuk mencatat jumlah pemanggilan fungsi rekursif pada algoritma seperti Merge Sort dan Quick Sort. Semakin besar jumlah recursive call, biasanya proses pembagian data juga semakin banyak. Kemudian pada baris [14], `array_accesses` digunakan untuk menghitung berapa kali algoritma mengakses array, khususnya pada algoritma seperti Counting Sort dan Radix Sort yang banyak melakukan pembacaan dan penulisan ulang data.

Pada baris [16], variabel `time_ms` digunakan untuk menyimpan waktu eksekusi algoritma dalam satuan milidetik. Variabel ini sangat penting untuk

membandingkan kecepatan masing-masing algoritma sorting pada berbagai kondisi data.

Bagian terakhir terdapat fungsi `reset()` pada baris [18] hingga [25]. Fungsi ini digunakan untuk mengembalikan seluruh nilai pengukuran menjadi 0 sebelum pengujian baru dimulai. Dengan adanya fungsi reset, setiap algoritma dapat diuji kembali tanpa membawa hasil pengukuran sebelumnya sehingga data pengujian tetap akurat dan rapi.

Pada file header **report.h** terdapat deklarasi fungsi yang digunakan untuk menampilkan hasil pengujian algoritma sorting ke layar. File ini berfungsi sebagai penghubung agar fungsi tampilan dapat dipanggil dari file lain tanpa perlu menuliskan ulang isi fungsi. Pada baris [1] digunakan `#pragma once` untuk memastikan file header hanya dibaca satu kali oleh compiler sehingga tidak terjadi duplikasi deklarasi fungsi.

Pada baris [2], program memanggil library vector. Walaupun pada potongan kode ini vector belum digunakan secara langsung, library tersebut biasanya disiapkan untuk kebutuhan pengolahan data dalam file implementasi terkait. Kemudian pada baris [3], file `metrics.h` dipanggil karena fungsi yang ada pada header ini menggunakan struct `Metrics` sebagai parameter utama untuk menampilkan hasil pengukuran algoritma.

Fungsi pertama dideklarasikan pada baris [5] yaitu `printMetrics(const Metrics& m);`. Fungsi ini digunakan untuk menampilkan seluruh hasil pengujian dari suatu algoritma sorting. Parameter `const Metrics& m` berarti fungsi menerima data `metrics` tanpa menyalin ulang isi struct sehingga penggunaan memori menjadi lebih efisien. Data yang biasanya ditampilkan melalui fungsi ini meliputi jumlah `comparison`, `swap`, `shift`, `recursion call`, `array access`, dan waktu eksekusi algoritma.

Kemudian pada baris [7] terdapat fungsi `printTableHeader();`. Fungsi ini digunakan untuk menampilkan bagian judul tabel sebelum hasil pengujian dicetak. Dengan adanya fungsi ini, tampilan output menjadi lebih rapi dan mudah dibaca karena setiap kolom seperti nama algoritma, jenis input, jumlah `comparison`, dan waktu eksekusi memiliki header yang jelas.

Secara keseluruhan, file header ini berfungsi untuk mengatur bagian tampilan hasil pengujian algoritma sorting. Pemisahan deklarasi fungsi seperti ini membuat struktur program menjadi lebih rapi, mudah dipahami, dan memudahkan pengelolaan kode ketika program semakin besar.

Pada file `reporter.cpp` ini terdapat implementasi fungsi yang digunakan untuk menampilkan hasil pengujian algoritma sorting dalam bentuk tabel yang rapi di terminal atau console. File ini bekerja sebagai bagian tampilan output sehingga hasil analisis seperti jumlah `comparison`, `swap`, dan waktu eksekusi dapat dibaca dengan lebih mudah.

Pada baris [1], file `reporter.h` dipanggil karena file ini merupakan implementasi dari deklarasi fungsi yang sudah dibuat sebelumnya pada header. Kemudian pada baris [2] digunakan library `iostream` untuk menampilkan teks ke layar menggunakan `std::cout`. Pada baris [3], program memanggil library `iomanip` yang berfungsi untuk mengatur tampilan output seperti lebar kolom dan jumlah angka desimal.

Baris [5] hingga [9] berisi konstanta ukuran kolom tabel. Misalnya `W_ALG` pada baris [5] digunakan untuk menentukan lebar kolom nama algoritma, `W_INPUT` pada baris [6] untuk jenis input data, dan `W_TIME` pada baris [9] untuk kolom waktu eksekusi. Dengan pengaturan ini, tampilan tabel menjadi lebih sejajar dan mudah dibaca walaupun isi datanya berbeda panjang.

Fungsi `printTableHeader()` dimulai pada baris [11]. Fungsi ini bertugas menampilkan bagian judul tabel sebelum hasil pengujian dicetak. Pada baris [12], digunakan `std::left` agar seluruh isi tabel rata kiri sehingga tampil lebih rapi. Selanjutnya pada baris [13] hingga [20], program menggunakan `std::setw()` untuk mengatur lebar setiap kolom sebelum mencetak judul seperti `Algorithm`, `Input Type`, `Comparisons`, `Swaps`, hingga `Time(ms)`. `std::setw(W_ALG)` pada baris [13] mengatur lebar kolom nama algoritma, `std::setw(W_STAT)` pada baris [16]-[19] mengatur lebar kolom statistik seperti `comparison` dan `swap`, sedangkan `std::setw(W_TIME)` pada baris [20] digunakan untuk kolom waktu eksekusi.

Pada baris [22], program mencetak garis pemisah menggunakan `std::string()` yang diisi karakter '-'. Garis ini berfungsi agar tampilan tabel terlihat lebih jelas dan terstruktur.

Fungsi berikutnya yaitu `printMetrics(const Metrics& m)` dimulai pada baris [25]. Fungsi ini digunakan untuk mencetak isi data hasil pengujian dari struct `Metrics`. Parameter `const Metrics& m` berarti data metrics dikirim tanpa disalin ulang sehingga lebih efisien.

Pada baris [26] hingga [35], setiap isi variabel dalam struct `Metrics` ditampilkan ke tabel menggunakan `std::setw()` agar posisinya tetap sejajar dengan header tabel. `m.algorithm_name` pada baris [27] menampilkan nama algoritma, `m.input_type` pada baris [28] menampilkan jenis input data, `m.comparisons` pada baris [30] menampilkan jumlah comparison, `m.swaps` pada baris [31] menampilkan

jumlah swap, dan `m.time_ms` pada baris [36] menampilkan waktu eksekusi algoritma.

Pada baris [35], digunakan `std::fixed` dan `std::setprecision(6)` untuk mengatur jumlah angka di belakang koma menjadi 6 digit. Hal ini membuat hasil waktu eksekusi tampil lebih presisi dan konsisten.

Pada file utama **main.cpp** program ini terdapat proses utama untuk menjalankan seluruh pengujian algoritma sorting secara otomatis. Program akan mencoba berbagai algoritma sorting menggunakan beberapa jenis data dan ukuran data, lalu hasil pengujiannya ditampilkan dalam bentuk tabel. Dengan struktur seperti ini, proses analisis performa algoritma menjadi lebih mudah dilakukan karena seluruh pengujian berjalan secara teratur dan konsisten.

Pada baris [1] hingga [4], program memanggil beberapa library bawaan C++ seperti `iostream`, `vector`, `functional`, dan `string`. Library tersebut digunakan untuk menampilkan output, menyimpan data dalam `vector`, menggunakan fungsi sebagai parameter, serta mengelola teks. Kemudian pada baris [6] hingga [9], program memanggil file header yang sebelumnya sudah dibuat, seperti `metrics.h`, `data_generator.h`, `sorting_algorithms.h`, dan `reporter.h`. File-file ini berisi fungsi dan struktur penting untuk proses pengujian algoritma sorting.

Baris [11] mendefinisikan `N_VALUES` yang berisi ukuran data yang akan diuji, yaitu 100, 1000, dan 10000. Dengan variasi ukuran data ini, program dapat membandingkan bagaimana performa algoritma berubah ketika jumlah data semakin besar.

Pada baris [13] hingga [19], dibuat daftar `INPUT_TYPES` yang berisi beberapa kondisi data untuk pengujian, yaitu:

- `RANDOM`
- `SORTED`
- `REVERSE_SORTED`
- `NEARLY_SORTED`
- `DUPLICATES`

Jenis data ini digunakan untuk melihat pengaruh bentuk data terhadap performa algoritma sorting.

Baris [21] mendefinisikan `SEED = 42` yang digunakan agar data random yang dihasilkan selalu konsisten pada setiap pengujian. Hal ini penting supaya hasil eksperimen lebih adil dan dapat dibandingkan secara akurat.

Pada baris [22], dibuat alias `SortFn` menggunakan `std::function`. Alias ini digunakan untuk menyimpan fungsi sorting dengan format parameter yang sama, yaitu menerima `vector<int>` dan `Metrics`.

Bagian penting berikutnya terdapat pada baris [24] hingga [31], yaitu daftar ALGORITHMS. Variabel ini menyimpan pasangan nama algoritma dan fungsi sorting yang akan dipanggil. Misalnya: "Bubble Sort" dipasangkan dengan fungsi `bubble_sort`, "Merge Sort" dipasangkan dengan `merge_sort`, "Radix Sort" dipasangkan dengan `radix_sort`.

Dengan cara ini, program dapat menjalankan seluruh algoritma menggunakan perulangan tanpa perlu menulis kode satu per satu.

Fungsi utama program dimulai pada baris [34] melalui `main()`. Pada baris [35] hingga [37], program menampilkan judul aplikasi ke layar agar output terlihat lebih jelas dan rapi.

Selanjutnya pada baris [39], program mulai melakukan perulangan untuk setiap algoritma sorting yang ada pada daftar ALGORITHMS. Variabel `name` menyimpan nama algoritma, sedangkan `sortFn` menyimpan fungsi sorting yang akan dijalankan. Pada baris [40], nama algoritma ditampilkan ke layar, lalu pada baris [41] fungsi `printTableHeader()` dipanggil untuk menampilkan header tabel hasil pengujian.

Baris [43] hingga [44] melakukan perulangan untuk setiap ukuran data dan setiap jenis input data. Artinya setiap algoritma akan diuji menggunakan seluruh kombinasi ukuran data dan kondisi data yang sudah ditentukan sebelumnya.

Pada baris [45], program membuat data pengujian menggunakan fungsi `generateData()`. Fungsi ini menghasilkan data sesuai ukuran dan jenis input yang dipilih, misalnya `random` atau `reverse sorted`.

Kemudian pada baris [47] hingga [50], dibuat objek `Metrics m` untuk menyimpan hasil pengukuran algoritma. Data seperti nama algoritma, jenis input, dan jumlah data disimpan ke dalam struct tersebut agar nantinya dapat ditampilkan pada tabel hasil pengujian.

Pada baris [52], fungsi sorting dijalankan melalui `sortFn(data, m);`. Di sinilah algoritma sorting benar-benar bekerja untuk mengurutkan data sekaligus mencatat statistik performanya seperti `comparison`, `swap`, dan waktu eksekusi.

Setelah proses sorting selesai, hasil pengukuran ditampilkan menggunakan `printMetrics(m);` pada baris [53]. Fungsi ini akan mencetak seluruh hasil pengujian dalam bentuk tabel yang rapi.

Pada baris [58], program menampilkan tulisan "Done." sebagai tanda bahwa seluruh proses pengujian telah selesai dilakukan, kemudian program berhenti pada baris [59] dengan return 0;

Pada file header **data_generator.h** berfungsi untuk mengatur proses pembuatan data pengujian pada algoritma sorting. File ini digunakan agar program dapat menghasilkan berbagai bentuk data seperti random, sorted, reverse sorted, nearly sorted, dan duplicates secara otomatis untuk keperluan analisis performa algoritma.

Pada baris [1], digunakan `#pragma once` untuk memastikan file header hanya dibaca satu kali oleh compiler sehingga tidak terjadi duplikasi deklarasi. Kemudian pada baris [2] dan [3], program memanggil library vector dan string. Library vector digunakan karena data pengujian disimpan dalam bentuk `std::vector<int>`, sedangkan string digunakan untuk mengubah tipe input menjadi teks yang dapat ditampilkan pada output program.

Bagian utama dimulai pada baris [5] dengan pembuatan enum class `InputType`. Enum ini digunakan untuk mendefinisikan beberapa jenis kondisi data yang akan dipakai dalam pengujian algoritma sorting. Dengan penggunaan enum, kode menjadi lebih rapi dan mudah dipahami karena setiap jenis data memiliki nama yang jelas.

Pada baris [6] terdapat `RANDOM` yang digunakan untuk menghasilkan data acak. Jenis data ini biasanya digunakan untuk melihat performa umum algoritma pada kondisi normal. Baris [7] berisi `SORTED` yang menghasilkan data sudah terurut menaik. Kondisi ini penting untuk menguji bagaimana algoritma bekerja ketika data sudah rapi.

Kemudian pada baris [8], terdapat `REVERSE_SORTED` yang menghasilkan data terurut menurun atau kebalikan dari urutan normal. Kondisi ini sering menjadi kasus terburuk bagi beberapa algoritma seperti Bubble Sort dan Insertion Sort karena banyak elemen harus dipindahkan.

Baris [9] berisi `NEARLY_SORTED` yang menghasilkan data hampir terurut. Biasanya hanya terdapat sedikit elemen yang salah posisi sehingga cocok untuk melihat performa algoritma pada data yang hampir rapi. Sedangkan pada baris [10], terdapat `DUPLICATES` yang menghasilkan banyak nilai kembar atau duplikat. Kondisi ini digunakan untuk menguji bagaimana algoritma menangani data dengan nilai yang sama dalam jumlah besar.

Pada baris [13], terdapat deklarasi fungsi `inputTypeName(InputType type);`. Fungsi ini digunakan untuk mengubah nilai enum seperti `RANDOM` atau `SORTED` menjadi teks biasa agar lebih mudah ditampilkan pada tabel hasil pengujian.

Selanjutnya pada baris [15], terdapat deklarasi fungsi `generateData()`. Fungsi ini berfungsi untuk membuat data pengujian sesuai ukuran dan jenis input yang dipilih. Parameter `n` digunakan untuk menentukan jumlah data, `type` menentukan bentuk data yang ingin dibuat, sedangkan `seed` digunakan agar data random yang dihasilkan tetap konsisten pada setiap pengujian. Dengan adanya fungsi ini, program dapat menghasilkan berbagai kondisi data secara otomatis tanpa perlu membuat data secara manual setiap kali pengujian dilakukan.

Pada file **data_generator.cpp** terdapat implementasi fungsi yang digunakan untuk membuat berbagai jenis data pengujian pada program sorting. File ini sangat penting karena seluruh eksperimen algoritma sorting bergantung pada data yang dihasilkan di sini. Dengan adanya file ini, program dapat membuat data random, sorted, reverse sorted, nearly sorted, dan duplicates secara otomatis sehingga pengujian menjadi lebih konsisten dan terstruktur.

Pada baris [1], file `data_generator.h` dipanggil karena file ini merupakan implementasi dari deklarasi fungsi yang sudah dibuat sebelumnya. Kemudian pada baris [2] hingga [5], program memanggil beberapa library tambahan seperti: `algorithm` untuk fungsi seperti `reverse()` dan `swap()`, `numeric` untuk fungsi `iota()`, `random` untuk menghasilkan angka acak, dan `stdexcept` untuk menangani error apabila input tidak valid.

Fungsi pertama dimulai pada baris [7] yaitu `inputTypeName(InputType type)`. Fungsi ini digunakan untuk mengubah nilai enum menjadi teks biasa agar lebih mudah ditampilkan pada tabel hasil pengujian. Pada baris [8] digunakan `switch` untuk memeriksa jenis input data yang diterima. Misalnya:

- `InputType::RANDOM` pada baris [9] dikembalikan menjadi teks "Random",
- `InputType::SORTED` pada baris [10] menjadi "Sorted",
- dan `InputType::DUPLICATES` pada baris [13] menjadi "Many Duplicates".

Jika jenis input tidak dikenali, maka pada baris [14] program mengembalikan teks "Unknown".

Fungsi utama pembuatan data dimulai pada baris [18] yaitu `generateData(int n, InputType type, unsigned int seed)`. Fungsi ini digunakan untuk membuat data sesuai ukuran dan jenis input yang dipilih. Parameter `n` menunjukkan jumlah data yang ingin dibuat, `type` menentukan bentuk data, dan `seed` digunakan agar data random yang dihasilkan tetap konsisten pada setiap pengujian.

Pada baris [19], program memeriksa apakah nilai `n` lebih kecil atau sama dengan 0. Jika iya, maka program akan menghasilkan error menggunakan `throw std::invalid_argument`. Hal ini dilakukan agar program tidak membuat data dengan ukuran yang tidak valid.

Baris [21] membuat objek `std::mt19937 rng(seed)` yang digunakan sebagai generator angka acak. Penggunaan seed membuat hasil data random tetap sama setiap kali program dijalankan sehingga hasil pengujian lebih adil dan mudah dibandingkan. Kemudian pada baris [22], dibuat vector data dengan ukuran n untuk menyimpan seluruh angka yang akan diuji.

Bagian utama berikutnya terdapat pada baris [24] yaitu `switch(type)`. Struktur ini digunakan untuk menentukan jenis data yang akan dibuat.

Pada baris [26] hingga [30], dibuat kondisi `RANDOM`. Program menggunakan `uniform_int_distribution` untuk menghasilkan angka acak dalam rentang 1 hingga $n * 10$. Kemudian pada baris [28], setiap elemen dalam vector diisi angka random menggunakan perulangan `for`.

Pada baris [32] hingga [35], dibuat kondisi `SORTED`. Program menggunakan fungsi `std::iota()` untuk mengisi vector dengan angka berurutan mulai dari 1. Hasilnya adalah data yang sudah terurut menaik.

Kemudian pada baris [37] hingga [41], dibuat kondisi `REVERSE_SORTED`. Program pertama-tama membuat data terurut menggunakan `iota()`, lalu pada baris [39] data dibalik menggunakan `std::reverse()`. Hasil akhirnya adalah data terurut menurun.

Pada baris [43] hingga [51], dibuat kondisi `NEARLY_SORTED`. Program awalnya membuat data terurut menggunakan `iota()`. Setelah itu pada baris [45], ditentukan jumlah swap sekitar 5% dari total data menggunakan $n / 20$. Kemudian pada baris [46], dibuat generator indeks acak. Selanjutnya pada baris [47]-[49], program menukar beberapa elemen secara acak menggunakan `std::swap()`. Hasil akhirnya adalah data yang sebagian besar sudah terurut namun masih memiliki sedikit posisi yang salah.

Pada baris [53] hingga [58], dibuat kondisi `DUPLICATES`. Program menentukan rentang angka yang lebih kecil menggunakan `range = n / 10`. Karena rentang nilainya kecil sementara jumlahnya besar, maka banyak angka yang akan muncul berulang. Setelah itu pada baris [56], setiap elemen diisi angka random dari rentang kecil tersebut sehingga menghasilkan banyak data duplikat. Pada baris [61], fungsi mengembalikan vector data yang sudah dibuat sesuai kondisi input yang dipilih.

SOAL 1

Untuk setiap algoritma, lakukan analisis terhadap aspek berikut:

Kompleksitas waktu:

- Best Case
- Average Case
- Worst Case

A. Bubble Sort

>> Bubble Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Bubble Sort	Random	100	4950	2564	0	0	0	0.025500
Bubble Sort	Sorted	100	4950	0	0	0	0	0.003100
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.037600
Bubble Sort	Nearly Sorted	100	4950	296	0	0	0	0.005100
Bubble Sort	Many Duplicates	100	4950	2315	0	0	0	0.015100
Bubble Sort	Random	1000	499500	243190	0	0	0	1.408100
Bubble Sort	Sorted	1000	499500	0	0	0	0	0.441800
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	2.724500
Bubble Sort	Nearly Sorted	1000	499500	25778	0	0	0	0.586900
Bubble Sort	Many Duplicates	1000	499500	240808	0	0	0	1.424000
Bubble Sort	Random	10000	49995000	25118148	0	0	0	149.790900
Bubble Sort	Sorted	10000	49995000	0	0	0	0	33.248400
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	266.830100
Bubble Sort	Nearly Sorted	10000	49995000	3119352	0	0	0	43.251300
Bubble Sort	Many Duplicates	10000	49995000	25093086	0	0	0	150.826500

Gambar 7 Hasil Pengujian Bubble Sort

Bubble Sort merupakan metode pengurutan data yang bekerja dengan cara membandingkan dua angka yang berdekatan kemudian menukarnya apabila posisinya salah. Proses ini dilakukan terus menerus dari awal hingga akhir data sampai seluruh angka berada dalam urutan yang benar. Pada setiap putaran, angka terbesar akan bergerak ke bagian akhir sehingga lama-kelamaan data menjadi terurut seluruhnya.

Pada (best case) kondisi terbaik, yaitu ketika data sudah terurut dari awal, Bubble Sort tidak perlu melakukan pertukaran data. Hal ini terlihat pada hasil pengujian bagian Sorted dimana jumlah Swaps bernilai 0 pada semua ukuran data. Waktu yang dibutuhkan juga lebih kecil dibanding kondisi lainnya. Meskipun demikian, algoritma masih tetap melakukan pengecekan pada seluruh data sehingga jumlah Comparisons tetap besar.

Pada (worst case) kondisi terburuk, yaitu ketika data tersusun terbalik atau Reverse Sorted, Bubble Sort harus melakukan pertukaran hampir pada setiap pengecekan. Hal ini terlihat dari jumlah Swaps yang sangat tinggi dan bahkan sama besar dengan jumlah Comparisons. Waktu eksekusi juga menjadi paling lama dibanding kondisi lainnya. Semakin banyak data yang digunakan, waktu proses meningkat sangat besar karena hampir semua elemen harus dipindahkan berulang kali.

Pada kondisi rata-rata atau Random, performa Bubble Sort berada di tengah-tengah antara kondisi terbaik dan terburuk. Algoritma masih melakukan banyak pengecekan dan pertukaran data, tetapi tidak sebanyak pada kondisi reverse sorted. Data *Many Duplicates* dan *Nearly Sorted* juga menunjukkan hasil

yang lebih baik dibanding random karena sebagian data sudah berada cukup dekat dengan posisi yang benar sehingga jumlah pertukaran lebih sedikit.

Bubble Sort sangat dipengaruhi oleh kondisi awal data. Pada data yang sudah terurut, waktu proses menjadi jauh lebih cepat karena tidak terjadi pertukaran. Sebaliknya pada data terbalik, waktu eksekusi meningkat drastis karena banyak perpindahan data yang harus dilakukan. Selain itu, semakin besar jumlah data, waktu proses Bubble Sort meningkat sangat tajam. Hal ini terlihat pada ukuran data 10000 dimana waktu eksekusi mencapai lebih dari 100 ms.

B. Selection Sort

>> Selection Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	95	0	0	0	0.007700
Selection Sort	Sorted	100	4950	0	0	0	0	0.006100
Selection Sort	Reverse Sorted	100	4950	50	0	0	0	0.009900
Selection Sort	Nearly Sorted	100	4950	4	0	0	0	0.006300
Selection Sort	Many Duplicates	100	4950	89	0	0	0	0.007700
Selection Sort	Random	1000	499500	995	0	0	0	0.782600
Selection Sort	Sorted	1000	499500	0	0	0	0	1.105500
Selection Sort	Reverse Sorted	1000	499500	500	0	0	0	0.781900
Selection Sort	Nearly Sorted	1000	499500	50	0	0	0	0.776500
Selection Sort	Many Duplicates	1000	499500	983	0	0	0	0.949100
Selection Sort	Random	10000	49995000	9992	0	0	0	85.182500
Selection Sort	Sorted	10000	49995000	0	0	0	0	84.525900
Selection Sort	Reverse Sorted	10000	49995000	5000	0	0	0	85.146000
Selection Sort	Nearly Sorted	10000	49995000	500	0	0	0	84.712600
Selection Sort	Many Duplicates	10000	49995000	9981	0	0	0	84.728800

Gambar 8 Hasil Pengujian Selection Sort

Selection Sort merupakan metode pengurutan data yang bekerja dengan cara mencari nilai paling kecil dari kumpulan data yang belum terurut, lalu menempatkannya ke posisi yang benar. Setelah satu nilai terkecil ditemukan dan dipindahkan, proses yang sama dilakukan kembali pada bagian data berikutnya sampai seluruh data tersusun rapi. Cara kerjanya cukup sederhana karena setiap putaran hanya fokus mencari satu nilai minimum.

Pada (best case) kondisi terbaik, yaitu ketika data sudah terurut dari awal, Selection Sort tetap melakukan pengecekan terhadap seluruh data. Hal ini terlihat dari jumlah Comparisons yang tetap besar meskipun nilai Swaps bernilai 0. Waktu eksekusi juga tidak jauh berbeda dengan kondisi lainnya karena algoritma tetap harus mencari nilai terkecil pada setiap putaran walaupun posisi data sebenarnya sudah benar.

Pada (worst case) kondisi terburuk, yaitu saat data tersusun terbalik atau Reverse Sorted, jumlah pertukaran data meningkat karena hampir setiap putaran menemukan nilai terkecil di posisi yang jauh dari tempat seharusnya. Hal ini terlihat dari jumlah Swaps yang lebih tinggi dibanding kondisi sorted. Namun waktu proses Selection Sort masih relatif stabil karena jumlah pengecekannya tetap hampir sama pada semua kondisi data.

Pada kondisi rata-rata atau Random, Selection Sort tetap melakukan banyak pengecekan untuk mencari nilai terkecil. Jumlah pertukaran data berada di antara kondisi terbaik dan terburuk. Pada data Nearly Sorted, jumlah pertukaran menjadi sangat sedikit karena sebagian besar data sudah mendekati posisi yang benar.

Sementara pada Many Duplicates, pertukaran masih cukup banyak tetapi tidak sebanyak pada data acak penuh.

Performa Selection Sort cukup stabil pada berbagai jenis data. Hal ini terlihat dari waktu eksekusi yang tidak mengalami perubahan terlalu besar antara data random, sorted, maupun reverse sorted. Jumlah Comparisons juga selalu sama karena algoritma tetap memeriksa seluruh bagian data untuk mencari nilai terkecil. Selain itu, semakin besar jumlah data maka waktu proses meningkat cukup drastis. Pada ukuran data 10000, waktu eksekusi mencapai lebih dari 80 ms.

C. Insertion Sort

>> Insertion Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Insertion Sort	Random	100	2656	0	2564	0	0	0.002900
Insertion Sort	Sorted	100	99	0	0	0	0	0.000200
Insertion Sort	Reverse Sorted	100	4950	0	4950	0	0	0.002600
Insertion Sort	Nearly Sorted	100	395	0	296	0	0	0.000500
Insertion Sort	Many Duplicates	100	2412	0	2315	0	0	0.001800
Insertion Sort	Random	1000	244181	0	243190	0	0	0.106400
Insertion Sort	Sorted	1000	999	0	0	0	0	0.001300
Insertion Sort	Reverse Sorted	1000	499500	0	499500	0	0	0.228600
Insertion Sort	Nearly Sorted	1000	26775	0	25778	0	0	0.020600
Insertion Sort	Many Duplicates	1000	241801	0	240808	0	0	0.116000
Insertion Sort	Random	10000	25128135	0	25118148	0	0	10.711900
Insertion Sort	Sorted	10000	9999	0	0	0	0	0.012500
Insertion Sort	Reverse Sorted	10000	49995000	0	49995000	0	0	24.324700
Insertion Sort	Nearly Sorted	10000	3129351	0	3119352	0	0	1.583400
Insertion Sort	Many Duplicates	10000	25103078	0	25093086	0	0	12.004100

Gambar 9 Hasil Pengujian Insertion Sort

Insertion Sort merupakan metode pengurutan data yang bekerja dengan cara mengambil satu data kemudian menyisipkannya ke posisi yang benar pada bagian data yang sudah terurut. Proses ini dilakukan sedikit demi sedikit dari kiri ke kanan sampai seluruh data tersusun rapi. Cara kerjanya mirip seperti menyusun kartu di tangan, dimana setiap kartu baru dimasukkan ke posisi yang sesuai.

Pada (best case) kondisi terbaik, yaitu ketika data sudah terurut dari awal (Sorted), Insertion Sort bekerja sangat cepat. Hal ini terlihat dari jumlah Comparisons dan Shifts yang sangat kecil dibanding kondisi lainnya. Pada ukuran data 10000 misalnya, jumlah comparison hanya sekitar 9999 dan shift bernilai 0. Waktu eksekusinya juga menjadi yang paling kecil karena algoritma tidak perlu memindahkan data.

Pada kondisi terburuk, yaitu ketika data tersusun terbalik (Reverse Sorted), Insertion Sort harus menggeser hampir seluruh data pada setiap langkah. Hal ini menyebabkan jumlah Shifts dan Comparisons meningkat sangat besar. Pada data ukuran 10000, jumlah shift mencapai hampir 50 juta dan waktu proses menjadi jauh lebih lama dibanding kondisi sorted. Semakin besar ukuran data, semakin besar pula peningkatan waktu eksekusinya.

Pada kondisi rata-rata atau Random, performa Insertion Sort berada di tengah antara kondisi terbaik dan terburuk. Algoritma masih melakukan cukup banyak pergeseran data, tetapi tidak sebanyak pada reverse sorted. Pada data Nearly Sorted, performanya jauh lebih baik karena sebagian besar data sudah berada dekat posisi yang benar sehingga perpindahan data menjadi lebih sedikit. Sementara pada

Many Duplicates, jumlah pergeseran juga lebih rendah dibanding random karena banyak nilai yang sama.

Insertion Sort sangat dipengaruhi oleh kondisi awal data. Ketika data sudah terurut atau hampir terurut, algoritma dapat bekerja dengan sangat cepat karena hanya sedikit perpindahan data yang diperlukan. Namun ketika data tersusun terbalik, waktu proses meningkat drastis akibat banyaknya pergeseran yang terjadi. Selain itu, semakin besar jumlah data maka waktu eksekusi juga meningkat cukup tajam, terutama pada kondisi reverse sorted.

D. Merge Sort

>> Merge Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Merge Sort	Random	100	535	0	0	199	0	0.013100
Merge Sort	Sorted	100	356	0	0	199	0	0.008700
Merge Sort	Reverse Sorted	100	316	0	0	199	0	0.008500
Merge Sort	Nearly Sorted	100	453	0	0	199	0	0.008400
Merge Sort	Many Duplicates	100	536	0	0	199	0	0.012800
Merge Sort	Random	1000	8690	0	0	1999	0	0.099400
Merge Sort	Sorted	1000	5044	0	0	1999	0	0.062800
Merge Sort	Reverse Sorted	1000	4932	0	0	1999	0	0.062300
Merge Sort	Nearly Sorted	1000	7628	0	0	1999	0	0.085700
Merge Sort	Many Duplicates	1000	8679	0	0	1999	0	0.093900
Merge Sort	Random	10000	120465	0	0	19999	0	1.015000
Merge Sort	Sorted	10000	69008	0	0	19999	0	0.769000
Merge Sort	Reverse Sorted	10000	64608	0	0	19999	0	0.943900
Merge Sort	Nearly Sorted	10000	111524	0	0	19999	0	0.712700
Merge Sort	Many Duplicates	10000	120461	0	0	19999	0	1.364500

Gambar 10 Hasil Pengujian Merge Sort

Merge Sort merupakan metode pengurutan data yang bekerja dengan cara membagi data menjadi bagian-bagian kecil, kemudian mengurutkannya dan menggabungkannya kembali menjadi satu urutan yang benar. Proses pembagian dilakukan terus menerus sampai data menjadi sangat kecil, lalu setiap bagian digabung kembali secara bertahap hingga seluruh data selesai diurutkan. Cara kerja ini membuat Merge Sort mampu menangani data dalam jumlah besar dengan cukup stabil.

Pada (best case), yaitu ketika data sudah terurut (Sorted), Merge Sort tetap melakukan proses pembagian dan penggabungan data seperti biasa. Walaupun demikian, jumlah Comparisons menjadi lebih sedikit dibanding kondisi random. Hal ini terlihat pada ukuran data 10000 dimana jumlah comparison lebih rendah dibanding beberapa kondisi lainnya. Waktu eksekusinya juga termasuk cepat dan stabil.

Pada (worst case), yaitu ketika data tersusun terbalik (Reverse Sorted), performa Merge Sort tidak mengalami penurunan yang terlalu besar. Jumlah comparison memang meningkat dibanding kondisi sorted, tetapi waktu proses tetap relatif mendekati kondisi lainnya. Hal ini menunjukkan bahwa susunan awal data tidak terlalu mempengaruhi cara kerja Merge Sort.

Pada kondisi rata-rata atau Random, Merge Sort tetap bekerja dengan cukup konsisten. Jumlah comparison dan waktu eksekusi berada di sekitar kondisi lainnya tanpa perbedaan yang terlalu jauh. Pada data Nearly Sorted, waktu proses bahkan sedikit lebih cepat karena beberapa bagian data sudah cukup dekat dengan urutan yang benar. Sementara pada Many Duplicates, jumlah comparison hampir sama

dengan data random karena proses pembagian dan penggabungan tetap berjalan seperti biasa.

Merge Sort memiliki performa yang sangat stabil pada semua kondisi data. Perbedaan waktu eksekusi antara data sorted, reverse sorted, random, maupun duplicates tidak terlalu besar. Selain itu, jumlah Rec.Calls selalu sama untuk ukuran data yang sama karena proses pembagian data dilakukan secara tetap. Ketika ukuran data meningkat dari 100 menjadi 10000, waktu proses memang bertambah, tetapi peningkatannya tidak sebesar Bubble Sort atau Insertion Sort.

E. Quick Sort

```
>> Quick Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	678	394	0	129	0	0.002300
Quick Sort	Sorted	100	4950	5049	0	199	0	0.003500
Quick Sort	Reverse Sorted	100	4950	2549	0	199	0	0.003500
Quick Sort	Nearly Sorted	100	2638	2607	0	191	0	0.003900
Quick Sort	Many Duplicates	100	909	236	0	181	0	0.002000
Quick Sort	Random	1000	10537	6125	0	1341	0	0.027700
Quick Sort	Sorted	1000	499500	500499	0	1999	0	0.349300
Quick Sort	Reverse Sorted	1000	499500	250499	0	1999	0	0.282900
Quick Sort	Nearly Sorted	1000	35189	25081	0	1611	0	0.041900
Quick Sort	Many Duplicates	1000	12685	4920	0	1801	0	0.036600
Quick Sort	Random	10000	147606	77996	0	13315	0	0.382400
Quick Sort	Sorted	10000	49995000	50004999	0	19999	0	32.773400
Quick Sort	Reverse Sorted	10000	49995000	25004999	0	19999	0	27.840300
Quick Sort	Nearly Sorted	10000	622473	458547	0	16511	0	0.693600
Quick Sort	Many Duplicates	10000	181016	67083	0	18003	0	0.302600

Gambar 11 Hasil Pengujian Quick Sort

Quick Sort merupakan metode pengurutan data yang bekerja dengan cara memilih satu nilai sebagai acuan, kemudian memisahkan data yang lebih kecil dan lebih besar dari nilai tersebut. Setelah data terbagi, proses yang sama dilakukan kembali pada bagian kiri dan kanan sampai seluruh data tersusun dengan benar. Cara kerja ini membuat Quick Sort mampu mengurutkan data dengan sangat cepat pada kondisi tertentu.

Pada kondisi terbaik, yaitu ketika pembagian data terjadi dengan seimbang, Quick Sort dapat bekerja sangat cepat. Hal ini biasanya terlihat pada data random dimana jumlah Comparisons, Swaps, dan waktu eksekusi jauh lebih kecil dibanding kondisi lainnya. Pada ukuran data 10000 misalnya, waktu proses random hanya sekitar 0.38 ms, jauh lebih cepat dibanding sorted maupun reverse sorted. Hal ini menunjukkan bahwa Quick Sort bekerja sangat baik ketika data terbagi secara merata.

Pada kondisi terburuk, yaitu ketika data sudah terurut (Sorted) atau terurut terbalik (Reverse Sorted), performa Quick Sort menurun sangat drastis. Hal ini terjadi karena pembagian data menjadi tidak seimbang sehingga proses pengulangan menjadi jauh lebih banyak. Pada hasil pengujian terlihat jumlah Comparisons, Swaps, dan Rec.Calls meningkat sangat besar. Waktu eksekusi juga menjadi paling lama dibanding kondisi lainnya. Pada ukuran data 10000, waktu proses bahkan mencapai lebih dari 27 ms hingga 32 ms.

Pada kondisi rata-rata atau Random, Quick Sort menunjukkan performa yang sangat baik. Jumlah perbandingan dan pertukaran data masih cukup rendah sehingga proses berjalan cepat. Pada data Nearly Sorted, performanya masih cukup

baik walaupun lebih lambat dibanding random karena sebagian data mulai menyebabkan pembagian yang kurang seimbang. Sementara pada Many Duplicates, waktu proses tetap tergolong cepat meskipun jumlah comparison dan swap masih cukup tinggi.

Berdasarkan hasil pengujian, terlihat bahwa performa Quick Sort sangat dipengaruhi oleh kondisi awal data. Pada data random, algoritma bekerja sangat cepat dan efisien bahkan untuk ukuran data besar. Namun pada data sorted dan reverse sorted, waktu proses meningkat sangat drastis karena pembagian data menjadi buruk. Selain itu, jumlah Rec.Calls juga meningkat besar pada kondisi tersebut. Walaupun memiliki kondisi terburuk yang cukup lambat, secara keseluruhan Quick Sort tetap menjadi salah satu algoritma sorting tercepat pada kondisi data normal atau acak.

F. Radix Sort

>> Radix Sort									
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)	
Radix Sort	Random	100	0	0	0	0	900	0.002700	
Radix Sort	Sorted	100	0	0	0	0	900	0.001800	
Radix Sort	Reverse Sorted	100	0	0	0	0	900	0.002200	
Radix Sort	Nearly Sorted	100	0	0	0	0	900	0.001800	
Radix Sort	Many Duplicates	100	0	0	0	0	600	0.001400	
Radix Sort	Random	1000	0	0	0	0	12000	0.015000	
Radix Sort	Sorted	1000	0	0	0	0	12000	0.013200	
Radix Sort	Reverse Sorted	1000	0	0	0	0	12000	0.016200	
Radix Sort	Nearly Sorted	1000	0	0	0	0	12000	0.013100	
Radix Sort	Many Duplicates	1000	0	0	0	0	9000	0.015300	
Radix Sort	Random	10000	0	0	0	0	150000	0.177400	
Radix Sort	Sorted	10000	0	0	0	0	150000	0.151800	
Radix Sort	Reverse Sorted	10000	0	0	0	0	150000	0.151500	
Radix Sort	Nearly Sorted	10000	0	0	0	0	150000	0.158400	
Radix Sort	Many Duplicates	10000	0	0	0	0	120000	0.125600	

Gambar 12 Hasil Pengujian Radix Sort

Radix Sort merupakan metode pengurutan data yang bekerja dengan cara mengelompokkan angka berdasarkan digitnya, dimulai dari digit paling belakang menuju digit paling depan. Algoritma ini tidak membandingkan angka satu per satu seperti Bubble Sort atau Quick Sort, tetapi langsung mengelompokkan angka sesuai nilai digit pada setiap tahap. Karena itu proses pengurutan dapat berjalan cepat dan stabil pada data angka dalam jumlah besar.

Pada kondisi terbaik, Radix Sort bekerja ketika jumlah digit pada data relatif sedikit sehingga proses pengelompokan digit tidak perlu dilakukan terlalu banyak kali. Dari hasil pengujian terlihat bahwa waktu eksekusi pada beberapa kondisi seperti Sorted dan Nearly Sorted cukup kecil. Selain itu nilai Arr.Access juga lebih rendah pada data Many Duplicates karena kemungkinan angka yang digunakan memiliki digit yang lebih pendek atau variasi digit yang lebih sedikit.

Pada kondisi terburuk, performa Radix Sort dipengaruhi oleh banyaknya digit angka yang harus diproses, bukan karena data terurut atau terbalik. Semakin panjang digit suatu angka dan semakin banyak kemiripan d, semakin banyak tahap pengelompokan yang dilakukan sehingga jumlah Arr.Access meningkat. Hal ini terlihat pada ukuran data 10000 dimana Arr.Access mencapai 150000 pada sebagian besar kondisi input. Walaupun demikian, peningkatan waktu proses masih tergolong stabil dibanding algoritma sorting lainnya.

Pada kondisi rata-rata atau Random, Radix Sort tetap menunjukkan performa yang konsisten. Hal ini terlihat dari perbedaan waktu eksekusi antar jenis input yang tidak terlalu jauh. Berbeda dengan algoritma lain, nilai Comparisons, Swaps, Shifts, dan Rec.Calls seluruhnya bernilai 0 pada semua pengujian. Hal ini

terjadi karena implementasi Radix Sort pada kode tidak menggunakan proses perbandingan data, pertukaran data langsung, pergeseran data, maupun rekursi. Aktivitas utama algoritma hanya terlihat pada `Arr.Access` karena proses sorting dilakukan melalui akses dan penyusunan ulang array berdasarkan digit angka.

Berdasarkan hasil pengujian, terlihat bahwa Radix Sort memiliki performa yang sangat stabil pada berbagai kondisi data seperti random, sorted, reverse sorted, nearly sorted, maupun many duplicates. Perbedaan waktu eksekusi antar kondisi sangat kecil dibanding algoritma lain seperti Bubble Sort atau Quick Sort. Selain itu, kenaikan waktu proses saat jumlah data bertambah juga masih cukup teratur. Faktor utama yang mempengaruhi performa Radix Sort adalah jumlah digit angka yang diproses dan kemiripan dari digit pada setiap data, bukan susunan awal data. Dari hasil tersebut dapat disimpulkan bahwa Radix Sort sangat cocok digunakan untuk pengurutan data angka dalam jumlah besar karena mampu bekerja cepat dan konsisten pada berbagai kondisi input data.

SOAL 2

Analisis Karakteristik pada setiap algoritma:

- Apakah algoritma bersifat stable atau tidak.
- Apakah algoritma termasuk in-place atau membutuhkan memori tambahan.
- Mekanisme utama algoritma dalam melakukan pengurutan

A. Bubble Sort

Bubble Sort merupakan algoritma pengurutan yang bekerja dengan cara membandingkan dua data yang posisinya berdekatan. Jika urutannya salah, maka kedua data tersebut ditukar. Proses ini dilakukan terus menerus dari awal hingga akhir data sampai semua angka tersusun dengan benar. Cara kerjanya menggeser angka terbesar sedikit demi sedikit ke bagian belakang pada setiap putaran.

Bubble Sort termasuk algoritma yang bersifat *stable*. Jika ada dua data yang nilainya sama, urutan asli kedua data tersebut tidak berubah setelah proses sorting selesai. Sebagai contoh, misalnya terdapat dua data dengan nilai 5 yang berasal dari posisi berbeda. Setelah diurutkan, kedua angka 5 tersebut akan tetap berada pada urutan yang sama seperti sebelumnya. Hal ini penting pada beberapa kondisi tertentu, misalnya ketika data memiliki informasi tambahan yang urutannya harus tetap dipertahankan.

Selain itu, Bubble Sort juga termasuk algoritma *in-place*. Artinya, proses pengurutan dilakukan langsung pada data utama tanpa membutuhkan tempat penyimpanan tambahan yang besar. Algoritma ini hanya memakai sedikit variabel sementara saat melakukan pertukaran data. Karena itu penggunaan memorinya kecil dan lebih hemat dibanding beberapa algoritma lain yang memerlukan array tambahan.

Mekanisme utama Bubble Sort adalah melakukan pengecekan berulang pada pasangan data yang berdekatan. Misalnya terdapat data:

5 3 8 1

Pada langkah pertama:

- 5 dibandingkan dengan 3 → ditukar menjadi 3 5 8 1
- 5 dibandingkan dengan 8 → tidak ditukar
- 8 dibandingkan dengan 1 → ditukar menjadi 3 5 1 8

Pada akhir putaran pertama, angka terbesar yaitu 8 sudah berada di posisi paling belakang. Proses ini terus diulang sampai semua angka berada pada posisi yang benar.

Untuk kompleksitas waktu, Bubble Sort termasuk algoritma yang cukup lambat jika digunakan pada jumlah data besar. Hal ini terjadi karena algoritma harus melakukan pengecekan berulang kali terhadap banyak pasangan data. Semakin

banyak jumlah data, semakin banyak juga proses perbandingan dan pertukaran yang dilakukan.

Pada kondisi terbaik atau *best case*, Bubble Sort dapat bekerja lebih cepat apabila data sudah terurut dari awal. Kompleksitas waktunya dapat mendekati:

$$O(n)$$

Artinya jumlah proses bertambah secara searah dengan jumlah data. Misalnya jika jumlah data dua kali lebih banyak, maka proses juga bertambah sekitar dua kali lipat. Kondisi ini biasanya terjadi apabila algoritma menggunakan optimasi untuk berhenti lebih awal ketika data sudah rapi. Namun pada kode pengujian yang digunakan, perulangan tetap berjalan penuh sehingga jumlah comparison masih tetap besar walaupun data sudah terurut.

Pada kondisi rata-rata maupun kondisi terburuk, kompleksitas waktunya menjadi:

$$O(n^2)$$

Artinya jumlah proses meningkat sangat cepat ketika jumlah data bertambah. Sebagai gambaran sederhana:

- jika data berjumlah 100, maka proses comparison dilakukan 4950
- jika data menjadi 1000, maka proses comparison dilakukan 499500

Inilah alasan mengapa Bubble Sort menjadi lambat pada data besar. Kondisi terburuk biasanya terjadi ketika data tersusun terbalik dari besar ke kecil karena hampir semua data harus dipindahkan berkali-kali.

Untuk kompleksitas ruang, Bubble Sort memiliki penggunaan memori yang sangat kecil yaitu:

$$O(1)$$

Artinya jumlah memori tambahan yang digunakan tetap kecil walaupun jumlah data semakin besar. Algoritma hanya membutuhkan beberapa variabel sementara untuk membantu proses pertukaran data.

Bubble Sort memiliki kelebihan berupa cara kerja yang sederhana, mudah dipahami, dan hemat memori. Namun kelemahannya adalah waktu proses menjadi sangat lambat ketika jumlah data besar karena terlalu banyak melakukan pengecekan dan pertukaran data.

B. Selection Sort

Selection Sort merupakan algoritma pengurutan yang bekerja dengan cara mencari nilai paling kecil dari bagian data yang belum terurut, kemudian menempatkannya ke posisi yang benar. Proses ini dilakukan berulang kali dari awal hingga akhir data sampai seluruh elemen tersusun secara berurutan. Pada setiap putaran, algoritma memilih satu nilai minimum lalu menukarnya dengan elemen pada posisi saat ini.

Selection Sort termasuk algoritma yang tidak *stable*. Hal ini karena proses pertukaran data dapat mengubah urutan asli elemen yang memiliki nilai sama. Sebagai contoh, apabila terdapat dua data dengan nilai yang sama tetapi berada pada posisi berbeda, proses swap dapat menyebabkan posisi keduanya berubah setelah sorting selesai. Oleh sebab itu urutan awal elemen yang sama tidak selalu dipertahankan.

Selection Sort juga termasuk algoritma *in-place* karena proses pengurutan dilakukan langsung pada array utama tanpa menggunakan tempat penyimpanan tambahan dalam jumlah besar. Algoritma hanya menggunakan beberapa variabel sederhana seperti indeks minimum dan variabel sementara saat pertukaran data dilakukan. Karena itu penggunaan memorinya relatif kecil.

Mekanisme utama Selection Sort adalah mencari nilai terkecil dari bagian data yang belum terurut. Setelah nilai terkecil ditemukan, elemen tersebut dipindahkan ke posisi yang benar melalui proses pertukaran data. Kemudian algoritma melanjutkan pencarian pada bagian data berikutnya sampai seluruh data selesai diurutkan. Sebagai contoh, tersedia data:

5 3 8 1

Langkah algoritma pada data sederhana di atas :

- algoritma mencari nilai terkecil yaitu 1
- lalu menukarnya dengan angka pertama,
- hasil menjadi:

1 3 8 5

Kemudian proses diulang pada sisa data sampai seluruh elemen tersusun.

Untuk kompleksitas waktu, Selection Sort memiliki jumlah proses yang relatif tetap pada berbagai kondisi data karena algoritma selalu melakukan pencarian nilai minimum pada seluruh bagian data yang belum terurut.

Pada kondisi terbaik, rata-rata, maupun terburuk, kompleksitas waktunya adalah:

$$O(n^2)$$

Hal ini terjadi karena algoritma menggunakan dua perulangan bersarang:

- perulangan pertama menentukan posisi elemen,
- perulangan kedua mencari nilai minimum.

Akibatnya jumlah comparison tetap besar walaupun kondisi data sudah terurut.

Berdasarkan hasil pengujian:

- Pada 100 data dilakukan 4950 comparison,
- Pada 1000 data dilakukan 499500 comparison,
- Pada 10000 data dilakukan 49995000 comparison.

Data tersebut menunjukkan bahwa jumlah comparison selalu sama untuk setiap jenis input seperti random, sorted, maupun reverse sorted. Hal ini membuktikan bahwa kondisi awal data tidak mempengaruhi jumlah pengecekan yang dilakukan oleh Selection Sort.

Meskipun jumlah comparison selalu besar, jumlah swap pada Selection Sort cenderung lebih sedikit dibanding Bubble Sort. Hal ini karena dalam satu putaran algoritma hanya melakukan maksimal satu kali pertukaran data setelah nilai minimum ditemukan. Untuk kompleksitas ruang, Selection Sort memiliki penggunaan memori sebesar:

$$O(1)$$

Karena hanya menggunakan sedikit variabel tambahan selama proses sorting berlangsung dan tidak membutuhkan array tambahan. Oleh sebab itu Selection Sort termasuk algoritma yang hemat memori, tetapi kurang efisien dalam waktu proses untuk data berukuran besar.

C. Insertion Sort

Insertion Sort merupakan algoritma pengurutan yang bekerja dengan cara mengambil satu elemen lalu menyisipkannya ke posisi yang sesuai pada bagian data yang sudah terurut. Proses ini dilakukan secara bertahap dari kiri ke kanan sampai seluruh data tersusun dengan benar. Pada setiap langkah, algoritma membandingkan elemen yang sedang diproses dengan elemen sebelumnya, kemudian menggeser data apabila posisinya belum sesuai.

Insertion Sort termasuk algoritma yang bersifat *stable*. Hal ini karena elemen yang memiliki nilai sama tidak akan saling bertukar posisi selama proses sorting berlangsung. Dengan demikian urutan awal data yang bernilai sama tetap dipertahankan setelah pengurutan selesai.

Insertion Sort juga termasuk algoritma *in-place* karena proses pengurutan dilakukan langsung pada array utama tanpa membutuhkan tempat penyimpanan tambahan dalam jumlah besar. Algoritma hanya menggunakan sedikit variabel tambahan seperti key untuk menyimpan sementara nilai yang sedang diproses. Oleh sebab itu penggunaan memorinya relatif kecil.

Mekanisme utama Insertion Sort adalah mengambil satu elemen dari bagian data yang belum terurut, kemudian menyisipkannya ke posisi yang benar pada bagian data yang sudah terurut. Jika terdapat elemen yang lebih besar, maka elemen tersebut digeser ke kanan sampai posisi yang sesuai ditemukan. Sebagai contoh:

5 3 8 1

Langkah pertama:

- angka 3 dibandingkan dengan 5,
- karena 3 lebih kecil, maka 5 digeser ke kanan,
- hasil menjadi:

3 5 8 1

Kemudian proses dilanjutkan pada angka berikutnya sampai seluruh data selesai diurutkan.

Untuk kompleksitas waktu, performa Insertion Sort sangat dipengaruhi oleh kondisi awal data. Pada kondisi terbaik atau *best case*, kompleksitas waktunya adalah:

$$O(n)$$

Kondisi ini terjadi ketika data sudah terurut. Pada keadaan tersebut, algoritma hanya perlu melakukan satu kali comparison untuk setiap elemen tanpa melakukan pergeseran data.

Berdasarkan hasil pengujian:

- Pada data 100 dilakukan 99 comparison,
- Pada data 1000 dilakukan 999 comparison,
- Pada data 10000 dilakukan 9999 comparison,
- Jumlah shift bernilai 0.

Data tersebut menunjukkan bahwa jumlah proses bertambah searah dengan jumlah data sehingga waktu eksekusi relatif cepat pada kondisi sorted.

Pada kondisi rata-rata dan kondisi terburuk, kompleksitas waktunya adalah:

$$O(n^2)$$

Kondisi terburuk terjadi ketika data tersusun terbalik. Dalam keadaan tersebut, setiap elemen harus dibandingkan dan digeser berkali-kali sampai mencapai posisi yang benar.

Berdasarkan hasil pengujian:

- Pada 10000 data reverse sorted dilakukan sekitar 49.995.000 comparison
- Jumlah shift mencapai sekitar 49.995.000.

Hal ini menunjukkan bahwa jumlah proses meningkat sangat besar ketika ukuran data bertambah dan kondisi data semakin tidak terurut.

Pada kondisi random, jumlah comparison dan shift berada di antara kondisi terbaik dan terburuk. Sedangkan pada nearly sorted, jumlah proses lebih kecil dibanding random karena sebagian besar data sudah berada dekat posisi yang benar.

Untuk kompleksitas ruang, Insertion Sort memiliki penggunaan memori sebesar:

$$O(1)$$

Karena hanya menggunakan sedikit variabel tambahan selama proses pengurutan berlangsung dan tidak membutuhkan array tambahan. Oleh sebab itu Insertion Sort termasuk algoritma yang hemat memori dan cukup efisien untuk data kecil atau data yang hampir terurut.

D. Merge Sort

Merge Sort merupakan algoritma pengurutan yang bekerja dengan cara membagi data menjadi beberapa bagian kecil, kemudian mengurutkannya dan menggabungkannya kembali hingga seluruh data tersusun dengan benar. Proses pembagian dilakukan terus menerus sampai ukuran data menjadi sangat kecil, kemudian setiap bagian digabung kembali secara bertahap dalam keadaan terurut.

Merge Sort termasuk algoritma yang bersifat **stable**. Hal ini karena elemen yang memiliki nilai sama tetap mempertahankan urutan awalnya setelah proses sorting selesai. Pada saat proses penggabungan data, elemen dengan nilai sama akan diambil sesuai urutan kemunculannya sehingga posisi relatif data tetap terjaga.

Merge Sort bukan algoritma **in-place** karena membutuhkan memori tambahan selama proses penggabungan data. Pada implementasi kode, algoritma membuat array sementara untuk menyimpan bagian kiri dan kanan data sebelum digabung kembali. Oleh sebab itu penggunaan memorinya lebih besar dibanding algoritma seperti Bubble Sort atau Insertion Sort yang bekerja langsung pada array utama.

Mekanisme utama Merge Sort adalah membagi data menjadi dua bagian, kemudian setiap bagian diproses kembali sampai tersisa satu elemen. Setelah itu data digabung kembali sambil dibandingkan satu per satu sehingga hasil akhirnya menjadi urut.

Sebagai contoh:

8 3 5 1

Data pertama kali dibagi menjadi:

8 3

5 1

Kemudian dibagi lagi:

8

3

5

1

Setelah itu data digabung kembali secara terurut:

3 8

1 5

Lalu digabung menjadi:

1 3 5 8

Untuk kompleksitas waktu, Merge Sort memiliki performa yang stabil pada semua kondisi data karena proses pembagian dan penggabungan tetap dilakukan dengan cara yang sama, baik data sudah terurut maupun belum.

Pada Merge Sort, proses pengurutan dilakukan dengan cara membagi data menjadi dua bagian secara terus menerus, kemudian setiap bagian digabung kembali dalam keadaan terurut. Karena proses pembagian dan penggabungan selalu dilakukan dengan pola yang sama, jumlah proses pada Merge Sort cenderung stabil pada berbagai kondisi data. Hal ini membuat performa Merge Sort tidak terlalu dipengaruhi oleh kondisi awal data seperti random, sorted, maupun reverse sorted.

Berdasarkan hasil pengujian, jumlah Rec.Calls selalu sama untuk ukuran data yang sama karena proses pembagian data berlangsung secara tetap. Selain itu, waktu eksekusi antara data random, sorted, reverse sorted, dan nearly sorted juga tidak memiliki perbedaan yang terlalu besar. Jumlah Comparisons meningkat secara bertahap seiring bertambahnya ukuran data, namun peningkatannya masih jauh lebih rendah dibanding algoritma seperti Bubble Sort atau Insertion Sort. Sebagai contoh, pada ukuran data 10000 jumlah comparison berada di kisaran sekitar 120 ribu hingga 127 ribu, sedangkan Bubble Sort dan Insertion Sort dapat mencapai puluhan juta comparison. Hasil tersebut menunjukkan bahwa Merge Sort lebih efisien untuk pengolahan data dalam jumlah besar.

Untuk kompleksitas ruang, Merge Sort memiliki penggunaan memori sebesar:

$$O(n)$$

Hal ini karena algoritma membutuhkan array tambahan untuk menyimpan sementara bagian kiri dan kanan data selama proses penggabungan berlangsung. Selain itu, proses rekursi juga menambah penggunaan memori selama algoritma dijalankan.

E. Quick Sort

Quick Sort merupakan algoritma pengurutan yang bekerja dengan cara memilih satu elemen sebagai titik acuan atau pivot, kemudian membagi data menjadi dua bagian, yaitu data yang lebih kecil dari pivot dan data yang lebih besar dari pivot. Setelah proses pembagian selesai, algoritma akan melakukan proses yang sama kembali pada bagian kiri dan kanan sampai seluruh data tersusun dengan benar. Cara kerja ini membuat Quick Sort mampu mengurutkan data dengan sangat cepat pada kondisi tertentu.

Quick Sort termasuk algoritma yang **tidak stable**. Hal ini karena selama proses pertukaran data, elemen yang memiliki nilai sama dapat berubah posisi. Akibatnya urutan awal data yang bernilai sama tidak selalu dipertahankan setelah proses sorting selesai.

Quick Sort termasuk algoritma **in-place** karena proses pengurutan dilakukan langsung pada array utama tanpa membutuhkan array tambahan dalam jumlah besar. Algoritma hanya menggunakan beberapa variabel tambahan untuk proses pembagian data dan rekursi. Namun karena menggunakan rekursi, tetap terdapat penggunaan memori tambahan pada stack pemanggilan fungsi.

Mekanisme utama Quick Sort adalah memilih satu elemen sebagai pivot, lalu memindahkan elemen yang lebih kecil ke bagian kiri dan elemen yang lebih besar ke bagian kanan. Setelah pivot berada pada posisi yang benar, proses yang sama dilakukan kembali pada bagian kiri dan kanan data sampai seluruh elemen selesai diurutkan.

Sebagai contoh:

8 3 5 1

Pivot yang akan dipilih adalah 1, maka seluruh data yang lebih kecil dari pivot akan berada di kiri dan yang lebih besar berada di kanan. Setelah proses pembagian selesai, algoritma akan melanjutkan pengurutan pada bagian data yang belum terurut sampai hasil akhirnya menjadi:

1 3 5 8

Untuk kompleksitas waktu, performa Quick Sort sangat dipengaruhi oleh keseimbangan proses pembagian data.

Pada kondisi terbaik atau **best case**, kompleksitas waktunya adalah:

$$O(n \log n)$$

Kondisi ini terjadi ketika pivot membagi data menjadi dua bagian yang relatif seimbang pada setiap proses rekursi. Dalam keadaan tersebut jumlah proses menjadi lebih sedikit sehingga waktu eksekusi berjalan sangat cepat.

Berdasarkan hasil pengujian:

- pada data random, waktu eksekusi Quick Sort termasuk sangat kecil,
- jumlah comparison dan recursive call meningkat secara lebih teratur,
- bahkan pada ukuran data 10000 waktu proses masih jauh lebih cepat dibanding Bubble Sort dan Insertion Sort.

Hal ini menunjukkan bahwa Quick Sort sangat efisien pada kondisi pembagian data yang seimbang. Pada kondisi terburuk atau **worst case**, kompleksitas waktunya menjadi:

$$O(n^2)$$

Kondisi ini terjadi ketika pivot selalu berada pada posisi paling kecil atau paling besar sehingga pembagian data menjadi tidak seimbang. Pada implementasi kode yang digunakan, pivot dipilih dari elemen terakhir array. Oleh sebab itu kondisi sorted dan reverse sorted menyebabkan proses pembagian menjadi buruk.

Berdasarkan hasil pengujian:

- jumlah Comparisons meningkat sangat besar pada data sorted dan reverse sorted,
- jumlah Rec.Calls juga meningkat drastis,
- waktu eksekusi menjadi jauh lebih lama dibanding data random.

Pada ukuran data 10000 data random memiliki waktu proses sekitar kurang dari 1 ms, sedangkan sorted dan reverse sorted mencapai puluhan ms. Hal ini menunjukkan bahwa performa Quick Sort dapat menurun sangat besar apabila pembagian data tidak seimbang.

Untuk kompleksitas ruang, Quick Sort memiliki penggunaan memori berbeda tergantung kondisi proses rekursinya.

Pada kondisi terbaik, kompleksitas ruangnya adalah

$$O(\log n)$$

karena kedalaman rekursi relatif kecil akibat pembagian data yang seimbang.

Sedangkan pada kondisi terburuk, kompleksitas ruangnya menjadi

$$O(n)$$

karena rekursi terjadi sangat dalam akibat pembagian data yang tidak seimbang.

Dari karakteristik tersebut, Quick Sort memiliki kelebihan berupa kecepatan yang sangat tinggi pada kondisi normal atau data acak. Namun performanya dapat menurun drastis pada kondisi tertentu apabila pemilihan pivot menghasilkan pembagian data yang tidak seimbang.

F. Radix Sort

Radix Sort merupakan algoritma pengurutan yang bekerja dengan cara mengelompokkan angka berdasarkan digitnya, dimulai dari digit paling belakang menuju digit paling depan. Algoritma ini tidak membandingkan angka secara langsung seperti Bubble Sort atau Quick Sort, melainkan menyusun data berdasarkan nilai digit pada setiap tahap. Setelah semua digit selesai diproses, data akan berada dalam urutan yang benar.

Radix Sort termasuk algoritma yang bersifat **stable**. Hal ini karena elemen yang memiliki nilai digit sama tetap mempertahankan urutan awalnya setelah proses pengelompokan dilakukan. Pada implementasi yang menggunakan Counting Sort sebagai proses bantu, urutan data dengan nilai sama tetap dijaga sehingga posisi relatif elemen tidak berubah.

Radix Sort bukan algoritma **in-place** karena membutuhkan memori tambahan selama proses sorting berlangsung. Pada implementasi kode, algoritma menggunakan array tambahan seperti output dan count untuk membantu proses pengelompokan digit. Oleh sebab itu penggunaan memorinya lebih besar dibanding algoritma seperti Bubble Sort atau Selection Sort yang bekerja langsung pada array utama.

Mekanisme utama Radix Sort adalah memproses angka berdasarkan posisi digit. Algoritma pertama-tama mengurutkan data berdasarkan digit satuan, kemudian digit puluhan, ratusan, dan seterusnya sampai seluruh digit selesai diproses.

Sebagai contoh terdapat data:

329 145 210 478

Tahap 1

Digit satuan:

- 329 → 9
- 145 → 5
- 210 → 0
- 478 → 8

Data kemudian disusun berdasarkan digit satuannya:

210 145 478 329

karena urutan digit satuannya adalah:

0 5 8 9

Tahap 2

Dari hasil sebelumnya:

210 145 478 329

Digit puluhan:

- 210 → 1
- 145 → 4
- 478 → 7
- 329 → 2

Kemudian data diurutkan kembali berdasarkan digit puluhan:

210 329 145 478

karena urutan digit puluhannya adalah:

1 2 4 7

Tahap 3

Dari hasil sebelumnya:

210 329 145 478

Digit ratusan:

- 210 → 2
- 329 → 3
- 145 → 1
- 478 → 4

Kemudian data diurutkan kembali berdasarkan digit ratusan:

145 210 329 478

Karena urutan digit ratusannya adalah:

1 2 3 4

Setelah seluruh digit selesai diproses dari belakang ke depan, data akhirnya menjadi terurut sepenuhnya.

Untuk kompleksitas waktu, performa Radix Sort dipengaruhi oleh jumlah digit yang dimiliki data, bukan oleh kondisi urutan data seperti sorted atau reverse sorted.

Pada kondisi terbaik atau **best case**, kompleksitas waktunya adalah:

$$O(d(n + k))$$

dengan:

- d = jumlah digit maksimum
- n = jumlah data
- k = jumlah kemungkinan digit

Kondisi terbaik terjadi ketika jumlah digit pada data relatif sedikit sehingga proses pengelompokan digit tidak perlu dilakukan terlalu banyak.

Berdasarkan hasil pengujian:

- waktu eksekusi antar kondisi data tidak memiliki perbedaan besar,
- nilai Comparisons, Swaps, Shifts, dan Rec.Calls selalu bernilai 0,
- aktivitas utama terlihat pada Arr.Access karena proses sorting dilakukan melalui akses array.

Hal ini menunjukkan bahwa performa Radix Sort relatif stabil pada berbagai kondisi input data. Namun, Pada kondisi terburuk, jumlah proses pada Radix Sort meningkat ketika data memiliki digit yang lebih panjang karena algoritma harus melakukan lebih banyak tahap pengelompokan digit. Setiap digit diproses satu per satu mulai dari digit paling belakang hingga digit paling depan. Oleh sebab itu, semakin banyak digit pada suatu angka, semakin banyak juga proses pengelompokan yang harus dilakukan. Berdasarkan hasil pengujian, ketika ukuran data meningkat dari 100 menjadi 10000, nilai Arr.Access juga meningkat sangat besar karena proses akses array dilakukan berulang pada setiap tahap digit. Meskipun demikian, peningkatan waktu proses masih terlihat cukup stabil dan tidak meningkat sedrastis algoritma dengan kompleksitas waktu $O(n^2)$ seperti Bubble Sort atau Selection Sort.

Hal ini menunjukkan bahwa performa Radix Sort lebih dipengaruhi oleh jumlah digit angka dibanding kondisi awal data. Untuk kompleksitas ruang, Radix Sort memiliki penggunaan memori sebesar:

$$O(n + k)$$

karena algoritma membutuhkan array tambahan untuk menyimpan hasil sementara dan proses penghitungan digit.

Dari karakteristik tersebut, Radix Sort memiliki kelebihan berupa performa yang stabil dan cepat untuk pengolahan data angka dalam jumlah besar. Namun algoritma ini membutuhkan memori tambahan dan hanya cocok digunakan pada data yang dapat diproses berdasarkan digit, seperti bilangan bulat.

SOAL 3

Analisis operasi:

- Jumlah comparison.
- Jumlah swap.
- Jumlah shift (khusus algoritma tertentu seperti Insertion Sort).
- Jumlah recursion call.
- Jumlah array accessed.
- Pengaruh bentuk data terhadap jumlah operasi.

A. Bubble Sort

Pada Bubble Sort, bentuk awal data sangat mempengaruhi jumlah operasi yang terjadi selama proses pengurutan. Hal ini terlihat jelas dari perbedaan jumlah Swap dan waktu eksekusi pada setiap jenis input data. Pada kondisi Sorted, data sudah berada dalam urutan yang benar sehingga tidak terjadi pertukaran elemen sama sekali. Akibatnya nilai Swap bernilai 0 pada seluruh ukuran data. Meskipun demikian, jumlah Comparison tetap sangat besar karena implementasi algoritma tetap melakukan seluruh proses pengecekan tanpa penghentian lebih awal. Sebaliknya pada kondisi Reverse Sorted, hampir setiap perbandingan menyebabkan pertukaran data sehingga jumlah swap menjadi yang paling tinggi. Sebagai contoh pada ukuran data 10000, jumlah swap mencapai sekitar 49.995.000 dan waktu eksekusi meningkat hingga sekitar 266 ms. Hal ini menunjukkan bahwa kondisi data yang terbalik membuat Bubble Sort bekerja jauh lebih berat.

Pada kondisi Random, jumlah swap berada di tengah karena sebagian data sudah dekat dengan posisi yang benar dan sebagian lainnya masih perlu dipindahkan. Sementara pada Nearly Sorted, jumlah swap jauh lebih kecil dibanding random karena sebagian besar data sudah hampir terurut. Pada data Many Duplicates, jumlah swap juga cukup besar, namun masih lebih rendah dibanding random dan reverse sorted karena banyak elemen memiliki nilai yang sama sehingga tidak selalu perlu ditukar.

Jumlah Comparison pada Bubble Sort selalu sama untuk ukuran data yang sama. Hal ini terjadi karena algoritma menggunakan dua perulangan bersarang yang tetap berjalan penuh sampai seluruh proses selesai. Berdasarkan hasil pengujian, pada ukuran data 100 dilakukan sekitar 4950 comparison, pada 1000 data sekitar 499500 comparison, dan pada 10000 data mencapai sekitar 49.995.000 comparison. Dari data tersebut terlihat bahwa jumlah comparison meningkat sangat cepat ketika ukuran data bertambah. Peningkatan inilah yang menyebabkan waktu eksekusi Bubble Sort menjadi sangat lambat pada data besar.

Jumlah Swap menjadi faktor utama yang membedakan performa antar kondisi data. Pada sorted, swap bernilai 0 karena tidak ada elemen yang perlu dipindahkan. Pada reverse sorted, hampir seluruh elemen harus dipindahkan berkali-kali sehingga swap mencapai nilai maksimum. Semakin besar jumlah swap,

semakin lama pula waktu proses yang dibutuhkan karena algoritma harus terus melakukan pertukaran posisi data.

Bubble Sort tidak menggunakan proses pergeseran data seperti Insertion Sort sehingga nilai Shift selalu 0 pada seluruh pengujian. Selain itu algoritma ini juga tidak menggunakan rekursi sehingga Rec.Calls bernilai 0. Pada implementasi kode yang digunakan, Arr.Access juga tidak dihitung sehingga nilainya tetap 0 walaupun sebenarnya algoritma tetap melakukan akses array saat membaca dan menukar data.

Dari keseluruhan hasil pengujian, terlihat bahwa kompleksitas waktu Bubble Sort pada kondisi rata-rata dan kondisi terburuk adalah:

$$O(n^2)$$

Karena jumlah comparison dan swap meningkat secara kuadrat terhadap jumlah data. Hal ini terlihat dari lonjakan waktu proses ketika ukuran data berubah dari 1000 menjadi 10000. Pada kondisi terbaik, kompleksitas waktunya sebenarnya dapat menjadi:

$$O(n)$$

Apabila algoritma menggunakan optimasi penghentian lebih awal ketika data sudah terurut. Namun pada implementasi pengujian ini, optimasi tersebut tidak digunakan sehingga jumlah comparison tetap besar walaupun data sudah rapi. Dari hasil tersebut dapat disimpulkan bahwa Bubble Sort mudah dipahami dan sederhana, tetapi kurang efisien untuk data berukuran besar karena jumlah operasinya meningkat sangat cepat ketika jumlah data bertambah.

B. Selection Sort

Pada Selection Sort, bentuk awal data tidak terlalu mempengaruhi jumlah proses utama yang dilakukan algoritma. Hal ini terlihat dari nilai Comparison yang selalu sama pada setiap jenis input dengan ukuran data yang sama. Baik data dalam kondisi Random, Sorted, Reverse Sorted, Nearly Sorted, maupun Many Duplicates, jumlah comparison tetap tidak berubah. Sebagai contoh, pada ukuran data 100 dilakukan sekitar 4950 comparison, pada 1000 data sekitar 499500 comparison, dan pada 10000 data mencapai sekitar 49.995.000 comparison. Hal ini terjadi karena Selection Sort tetap mencari nilai terkecil pada seluruh bagian data yang belum terurut pada setiap putaran, tanpa memperhatikan apakah data sudah rapi atau belum.

Walaupun, jumlah comparison selalu sama, bentuk data tetap mempengaruhi jumlah Swap dan waktu eksekusi. Pada kondisi Sorted, swap bernilai 0 karena seluruh elemen sudah berada di posisi yang benar sehingga tidak diperlukan pertukaran data. Pada kondisi Nearly Sorted, jumlah swap juga sangat kecil karena hanya sedikit elemen yang perlu dipindahkan. Sebaliknya pada Random, Reverse Sorted, dan Many Duplicates, jumlah swap meningkat karena lebih banyak elemen yang berada di posisi yang salah. Namun jumlah swap pada Selection Sort tetap jauh lebih kecil dibanding Bubble Sort karena algoritma ini hanya melakukan maksimal satu pertukaran pada setiap putaran setelah nilai minimum ditemukan.

Sebagai contoh pada ukuran data 10000:

- Random menghasilkan sekitar 9992 swap,
- Reverse Sorted sekitar 5000 swap,
- Many Duplicates sekitar 9981 swap,
- Sorted tetap 0 swap.

Data tersebut menunjukkan bahwa jumlah swap pada Selection Sort meningkat secara lebih stabil dan tidak sebesar Bubble Sort yang dapat mencapai puluhan juta swap.

Selection Sort tidak menggunakan proses pergeseran data sehingga nilai Shift selalu 0 pada seluruh pengujian. Selain itu algoritma ini juga tidak menggunakan rekursi sehingga Rec.Calls bernilai 0. Pada implementasi kode yang

digunakan, Arr.Access juga tidak dihitung sehingga nilainya tetap 0 walaupun sebenarnya algoritma tetap melakukan akses array saat membaca dan menukar data.

Dari hasil pengujian terlihat bahwa waktu eksekusi Selection Sort meningkat cukup besar ketika ukuran data bertambah. Hal ini terutama disebabkan oleh jumlah comparison yang selalu sangat besar pada setiap kondisi data. Kompleksitas waktu Selection Sort pada kondisi terbaik, rata-rata, maupun terburuk tetap sama yaitu:

$$O(n^2)$$

Karena algoritma tetap melakukan pencarian nilai minimum pada seluruh bagian data yang belum terurut. Berbeda dengan Bubble Sort atau Insertion Sort yang dapat menjadi lebih cepat pada data yang hampir terurut, performa Selection Sort cenderung tetap karena jumlah comparison tidak berubah. Dari hasil tersebut dapat disimpulkan bahwa Selection Sort memiliki penggunaan swap yang lebih sedikit dan penggunaan memori yang kecil, tetapi masih kurang efisien untuk data berukuran besar karena jumlah comparison meningkat sangat cepat seiring bertambahnya jumlah data.

C. Insertion Sort

Pada Insertion Sort, bentuk awal data sangat mempengaruhi jumlah operasi yang dilakukan selama proses pengurutan. Hal ini terjadi karena algoritma bekerja dengan cara menyisipkan elemen ke posisi yang benar pada bagian data yang sudah terurut. Jika data sudah hampir rapi, proses perpindahan data menjadi jauh lebih sedikit. Sebaliknya jika data tersusun terbalik, hampir seluruh elemen harus digeser berulang kali sehingga jumlah operasi meningkat sangat besar.

Pengaruh bentuk data terlihat jelas pada jumlah Comparison dan Shift. Pada kondisi Sorted, jumlah comparison sangat kecil karena setiap elemen langsung berada pada posisi yang benar sehingga algoritma hanya melakukan satu pengecekan sederhana pada setiap langkah. Sebagai contoh, pada ukuran data 100 hanya dilakukan 99 comparison, pada 1000 data sekitar 999 comparison, dan pada 10000 data sekitar 9999 comparison. Selain itu jumlah Shift bernilai 0 karena tidak ada elemen yang perlu dipindahkan. Akibatnya waktu eksekusi menjadi sangat cepat.

Sebaliknya pada kondisi Reverse Sorted, jumlah comparison dan shift meningkat sangat besar karena setiap elemen harus dibandingkan dan digeser hingga ke bagian depan array. Pada ukuran data 10000, jumlah comparison mencapai sekitar 49.995.000 dan jumlah shift juga mencapai sekitar 49.995.000. Hal ini membuat waktu eksekusi meningkat drastis hingga sekitar 24 ms. Dari data tersebut terlihat bahwa kondisi data yang terbalik merupakan kondisi paling berat bagi Insertion Sort.

Pada kondisi Random, jumlah comparison dan shift berada di tengah karena sebagian elemen perlu dipindahkan dan sebagian lainnya sudah cukup dekat dengan posisi yang benar. Sedangkan pada Nearly Sorted, jumlah operasi jauh lebih kecil dibanding random karena sebagian besar data sudah hampir terurut. Pada Many Duplicates, jumlah comparison dan shift masih cukup besar, namun sedikit lebih rendah dibanding random karena banyak elemen memiliki nilai yang sama sehingga perpindahan data tidak selalu diperlukan.

Berbeda dengan Bubble Sort atau Selection Sort, Insertion Sort tidak menggunakan proses Swap. Pada implementasi kode yang digunakan, perpindahan

data dilakukan dengan cara menggeser elemen ke kanan sehingga nilai Swap selalu 0.

Operasi utama pada algoritma ini justru terlihat pada bagian Shift karena setiap elemen yang dipindahkan dihitung sebagai pergeseran data.

Insertion Sort juga tidak menggunakan rekursi sehingga Rec.Calls bernilai 0 pada seluruh pengujian. Selain itu Arr.Access tidak dihitung pada implementasi kode sehingga nilainya tetap 0 walaupun algoritma tetap melakukan akses terhadap array selama proses penyisipan data berlangsung.

Dari keseluruhan hasil pengujian terlihat bahwa kompleksitas waktu Insertion Sort sangat dipengaruhi oleh kondisi awal data. Pada kondisi terbaik atau ketika data sudah terurut, kompleksitas waktunya adalah:

$$O(n)$$

Karena algoritma hanya melakukan sedikit comparison tanpa pergeseran data.

Sedangkan pada kondisi rata-rata dan kondisi terburuk, kompleksitas waktunya menjadi:

$$O(n^2)$$

Karena jumlah comparison dan shift meningkat sangat besar ketika data semakin tidak terurut. Dari hasil pengujian dapat disimpulkan bahwa Insertion Sort sangat efisien untuk data kecil atau data yang sudah hampir terurut, namun performanya menurun cukup drastis pada data besar yang tidak terurut.

D. Merge Sort

Pada Merge Sort, bentuk awal data tidak terlalu mempengaruhi jumlah operasi yang dilakukan algoritma. Hal ini terjadi karena Merge Sort selalu membagi data menjadi beberapa bagian kecil lalu menggabungkannya kembali dengan cara yang sama, baik data dalam keadaan random, sorted, reverse sorted, maupun nearly sorted. Akibatnya performa algoritma cenderung stabil pada berbagai kondisi data dan tidak mengalami perbedaan yang terlalu ekstrem seperti Bubble Sort atau Insertion Sort.

Pengaruh bentuk data tetap terlihat pada jumlah Comparison, namun perbedaannya tidak terlalu besar. Pada kondisi Sorted dan Reverse Sorted, jumlah comparison justru sedikit lebih kecil dibanding Random atau Many Duplicates. Sebagai contoh pada ukuran data 10000:

- Random menghasilkan sekitar 120465 comparison,
- Sorted sekitar 69008 comparison,
- Reverse Sorted sekitar 64608 comparison,
- Nearly Sorted mencapai sekitar 111524 comparison.

Data tersebut menunjukkan bahwa jumlah comparison pada Merge Sort tetap meningkat ketika ukuran data bertambah, tetapi peningkatannya jauh lebih rendah dibanding algoritma dengan kompleksitas $O(n^2)$. Hal ini membuat waktu eksekusi Merge Sort tetap relatif stabil walaupun jumlah data menjadi besar.

Pada implementasi Merge Sort yang digunakan, nilai Swap selalu 0 karena algoritma tidak melakukan pertukaran data secara langsung antar elemen. Proses pengurutan dilakukan dengan cara menggabungkan kembali bagian data yang sudah dipisahkan sebelumnya. Selain itu Merge Sort juga tidak menggunakan proses pergeseran data seperti Insertion Sort sehingga nilai Shift tetap 0 pada seluruh pengujian.

Salah satu operasi utama pada Merge Sort terlihat pada bagian Rec.Calls. Nilai ini cukup besar karena algoritma bekerja menggunakan rekursi untuk membagi data menjadi bagian-bagian kecil. Semakin besar jumlah data, semakin banyak pemanggilan fungsi rekursif yang dilakukan. Sebagai contoh:

- Pada ukuran data 100 terdapat sekitar 199 recursive call,
- Pada 1000 data sekitar 1999 recursive call,

- Pada 10000 data mencapai sekitar 19999 recursive call.

Hal ini menunjukkan bahwa jumlah rekursi meningkat secara teratur mengikuti penambahan ukuran data. Pada implementasi kode yang digunakan, Arr.Access tidak dihitung sehingga nilainya tetap 0 walaupun sebenarnya Merge Sort tetap melakukan akses array selama proses pembagian dan penggabungan data berlangsung.

Dari keseluruhan hasil pengujian terlihat bahwa kompleksitas waktu Merge Sort pada kondisi terbaik, rata-rata, maupun terburuk tetap sama yaitu:

$$O(n \log n)$$

Hal ini terjadi karena proses pembagian dan penggabungan data selalu dilakukan dengan pola yang sama tanpa dipengaruhi secara besar oleh bentuk awal data. Dibanding algoritma seperti Bubble Sort atau Selection Sort, jumlah comparison Merge Sort jauh lebih kecil ketika ukuran data sangat besar. Sebagai contoh pada ukuran data 10000, comparison Merge Sort hanya berada di kisaran ratusan ribu, , sedangkan Bubble Sort dapat mencapai puluhan juta comparison.

E. Quick Sort

Pada Quick Sort, bentuk awal data sangat mempengaruhi jumlah operasi dan waktu eksekusi algoritma. Hal ini terjadi karena Quick Sort bekerja dengan memilih satu elemen sebagai pivot untuk membagi data menjadi dua bagian. Jika pembagian data terjadi dengan seimbang, proses sorting berjalan sangat cepat. Namun jika pembagian tidak seimbang, jumlah operasi meningkat sangat besar dan performa algoritma menurun drastis.

Pengaruh bentuk data terlihat jelas pada jumlah Comparison, Swap, dan Rec.Calls. Pada kondisi Random, jumlah comparison relatif kecil dibanding kondisi sorted atau reverse sorted. Sebagai contoh pada ukuran data 10000, data random menghasilkan sekitar 147666 comparison dengan waktu proses sekitar 0.38 ms. Hal ini menunjukkan bahwa pembagian data berlangsung cukup seimbang sehingga proses sorting berjalan efisien.

Sebaliknya pada kondisi Sorted dan Reverse Sorted, jumlah comparison meningkat sangat besar hingga sekitar 49.995.000 comparison. Waktu eksekusinya juga meningkat drastis menjadi sekitar 27 ms hingga 32 ms. Kondisi ini terjadi karena implementasi Quick Sort pada kode menggunakan elemen terakhir sebagai pivot. Ketika data sudah terurut atau terbalik, pivot selalu berada pada posisi yang buruk sehingga pembagian data menjadi tidak seimbang. Akibatnya jumlah proses rekursi dan comparison meningkat sangat besar. Pada kondisi Nearly Sorted, performa Quick Sort masih lebih baik dibanding sorted penuh, tetapi jumlah operasi tetap cukup tinggi karena sebagian pembagian data masih belum seimbang. Sementara pada Many Duplicates, jumlah comparison dan swap lebih rendah dibanding random karena banyak elemen memiliki nilai yang sama sehingga perpindahan data tidak sebanyak kondisi lainnya.

Jumlah Swap pada Quick Sort cukup besar karena algoritma terus menukar posisi elemen selama proses partition berlangsung. Pada kondisi sorted, jumlah swap bahkan lebih besar dibanding reverse sorted. Sebagai contoh pada ukuran data 10000:

- Sorted menghasilkan sekitar 5.000.499 swap,
- Reverse Sorted sekitar 2.500.499 swap,

- Random sekitar 77.996 swap.

Hal ini menunjukkan bahwa kondisi data sangat mempengaruhi banyaknya perpindahan elemen pada Quick Sort.

Quick Sort tidak menggunakan proses pergeseran data seperti Insertion Sort sehingga nilai Shift selalu 0 pada seluruh pengujian. Namun algoritma ini menggunakan rekursi secara intensif sehingga nilai Rec.Calls cukup besar. Pada ukuran data 10000:

- Random menghasilkan sekitar 13315 recursive call,
- Sorted dan Reverse Sorted mencapai sekitar 19999 recursive call.

Jumlah rekursi yang tinggi pada sorted dan reverse sorted menunjukkan bahwa pembagian data menjadi sangat tidak seimbang sehingga proses pemanggilan fungsi berlangsung jauh lebih banyak.

Pada implementasi kode yang digunakan, Arr.Access tidak dihitung sehingga nilainya tetap 0 walaupun sebenarnya Quick Sort tetap melakukan akses array selama proses comparison dan partition berlangsung. Dari keseluruhan hasil pengujian terlihat bahwa kompleksitas waktu Quick Sort sangat bergantung pada kualitas pembagian data. Pada kondisi terbaik dan rata-rata, kompleksitas waktunya adalah:

$$O(n \log n)$$

Karena data terbagi cukup seimbang sehingga jumlah proses meningkat secara lebih stabil.

Sedangkan pada kondisi terburuk, kompleksitas waktunya menjadi:

$$O(n^2)$$

Karena pembagian data menjadi sangat tidak seimbang dan menyebabkan jumlah comparison, swap, serta recursive call meningkat drastis.

F. Radix Sort

Pada Radix Sort, bentuk awal data tidak terlalu mempengaruhi jumlah operasi yang dilakukan selama proses pengurutan. Hal ini berbeda dengan Bubble Sort atau Quick Sort yang sangat dipengaruhi kondisi data seperti sorted atau reverse sorted. Radix Sort bekerja berdasarkan digit angka, bukan berdasarkan perbandingan antar elemen. Oleh sebab itu jumlah proses lebih dipengaruhi oleh banyaknya digit dan jumlah data dibanding urutan awal data.

Berdasarkan hasil pengujian, nilai Comparison, Swap, Shift, dan Rec.Calls selalu bernilai 0 pada seluruh kondisi data. Hal ini terjadi karena Radix Sort memang tidak menggunakan proses perbandingan antar elemen seperti algoritma sorting lainnya. Algoritma ini juga tidak melakukan pertukaran data secara langsung antar elemen sehingga nilai swap tetap 0. Selain itu Radix Sort tidak menggunakan proses pergeseran data seperti Insertion Sort dan tidak menggunakan rekursi sehingga nilai shift dan recursion call juga tetap 0 pada seluruh pengujian.

Operasi utama pada Radix Sort terlihat pada bagian Arr.Access. Nilai ini menunjukkan seberapa sering algoritma mengakses array selama proses pengelompokan digit berlangsung. Pada implementasi kode, setiap tahap sorting digit melakukan pembacaan dan penulisan ulang data ke array sementara sehingga jumlah array access menjadi sangat besar. Sebagai contoh:

- Pada ukuran data 100, nilai Arr.Access berada di kisaran 600 hingga 900,
- Pada ukuran data 1000 meningkat menjadi sekitar 9000 hingga 12000,
- Pada ukuran data 10000 mencapai sekitar 120000 hingga 150000.

Perbedaan nilai tersebut dipengaruhi oleh jumlah digit angka yang diproses. Data dengan digit lebih panjang membutuhkan lebih banyak tahap pengelompokan sehingga jumlah akses array meningkat. Hal ini terlihat pada kondisi Many Duplicates yang memiliki nilai Arr.Access lebih rendah dibanding kondisi lainnya karena kemungkinan jumlah digit yang diproses lebih sedikit.

Walaupun jumlah array access cukup besar, waktu eksekusi Radix Sort tetap relatif stabil dan sangat cepat. Pada ukuran data 10000, waktu proses hanya berada di sekitar 0.12 ms hingga 0.17 ms. Perbedaan waktu antar kondisi data juga sangat kecil. Hal ini menunjukkan bahwa performa Radix Sort tidak terlalu dipengaruhi

bentuk awal data, melainkan lebih dipengaruhi jumlah digit pada angka yang diproses.

Kompleksitas waktu Radix Sort adalah:

$$O(d(n + k))$$

dengan:

- d sebagai jumlah digit maksimum,
- n sebagai jumlah data,
- k sebagai jumlah kemungkinan digit.

Karena jumlah digit biasanya tidak terlalu besar, Radix Sort mampu bekerja sangat cepat pada data angka dalam jumlah besar. Selain itu kompleksitas waktunya relatif stabil pada kondisi terbaik, rata-rata, maupun terburuk karena proses pengelompokan digit selalu dilakukan dengan pola yang sama.

Untuk penggunaan memori, Radix Sort membutuhkan array tambahan selama proses sorting berlangsung sehingga algoritma ini tidak termasuk in-place. Kompleksitas ruangnya adalah:

$$O(n + k)$$

Karena algoritma memerlukan array sementara dan array penghitung digit untuk membantu proses pengurutan.

Dari hasil pengujian dapat disimpulkan bahwa Radix Sort sangat efisien untuk pengolahan data angka dalam jumlah besar karena waktu prosesnya stabil dan tidak dipengaruhi secara besar oleh kondisi awal data.

SOAL 4

Analisis performa:

- Perbandingan waktu eksekusi antar algoritma.
- Algoritma mana yang paling cepat untuk dataset kecil dan besar.
- Pengaruh distribusi data terhadap performa algoritma.
- Hubungan antara teori kompleksitas dan hasil eksperimen yang diperoleh.

Berdasarkan hasil pengujian yang diperoleh, terlihat bahwa setiap algoritma sorting memiliki performa yang berbeda tergantung ukuran data dan kondisi dataset yang digunakan. Untuk dataset kecil seperti 100 data, hampir seluruh algoritma memiliki waktu eksekusi yang sangat cepat sehingga perbedaannya tidak terlalu terlihat. Namun ketika ukuran data meningkat menjadi 1000 hingga 10000, perbedaan performa antar algoritma mulai terlihat dengan jelas. Algoritma sederhana seperti Bubble Sort, Selection Sort, dan Insertion Sort mengalami peningkatan waktu eksekusi yang sangat besar karena jumlah operasi meningkat drastis. Sebaliknya Merge Sort, Quick Sort, dan Radix Sort tetap mampu mempertahankan waktu proses yang relatif cepat meskipun jumlah data bertambah besar.

Berdasarkan hasil eksperimen, Radix Sort menjadi algoritma dengan performa paling cepat pada hampir seluruh pengujian data besar. Hal ini terlihat dari waktu eksekusinya yang tetap sangat kecil bahkan pada ukuran data 10000. Merge Sort dan Quick Sort juga menunjukkan performa yang sangat baik pada dataset besar karena jumlah operasi yang dilakukan jauh lebih sedikit dibanding algoritma sederhana. Sementara itu Bubble Sort menjadi algoritma paling lambat terutama pada kondisi reverse sorted karena jumlah comparison dan swap meningkat hingga puluhan juta operasi. Selection Sort juga memiliki performa lambat karena jumlah comparison selalu besar walaupun jumlah swap relatif sedikit. Insertion Sort berada di tengah karena mampu bekerja sangat cepat pada data yang sudah hampir terurut, namun performanya menurun cukup drastis pada data random dan reverse sorted.

Distribusi atau bentuk awal data juga memberikan pengaruh besar terhadap performa beberapa algoritma. Bubble Sort dan Insertion Sort sangat dipengaruhi oleh kondisi data. Ketika data sudah terurut atau nearly sorted, jumlah operasi menjadi jauh lebih sedikit sehingga waktu proses lebih cepat. Sebaliknya pada reverse sorted, kedua algoritma tersebut harus melakukan banyak perpindahan data sehingga waktu eksekusi meningkat drastis. Quick Sort juga sangat dipengaruhi distribusi data karena pemilihan pivot pada implementasi kode menggunakan elemen terakhir array. Akibatnya kondisi sorted dan reverse sorted menghasilkan pembagian data yang buruk dan menyebabkan jumlah comparison serta recursion

call meningkat sangat besar. Berbeda dengan ketiga algoritma tersebut, Merge Sort dan Radix Sort cenderung lebih stabil karena mekanisme kerjanya tidak terlalu dipengaruhi urutan awal data.

Hasil eksperimen yang diperoleh juga sesuai dengan teori kompleksitas waktu masing-masing algoritma. Bubble Sort, Selection Sort, dan Insertion Sort memiliki kompleksitas waktu:

$$O(n^2)$$

sehingga ketika ukuran data meningkat, jumlah operasi bertambah sangat cepat dan menyebabkan waktu eksekusi meningkat drastis. Hal ini terlihat jelas pada pengujian data 10000 yang menghasilkan waktu proses jauh lebih besar dibanding algoritma lainnya.

Merge Sort dan Quick Sort pada kondisi normal memiliki kompleksitas waktu:

$$O(n \log n)$$

karena proses pengurutan dilakukan dengan pembagian data yang lebih efisien. Oleh sebab itu waktu eksekusinya tetap relatif cepat walaupun jumlah data besar. Namun pada Quick Sort, hasil eksperimen juga menunjukkan kondisi worst case yang sesuai teori ketika data sorted atau reverse sorted menyebabkan kompleksitas waktunya berubah menjadi:

$$O(n^2)$$

karena pembagian data menjadi tidak seimbang.

Sedangkan Radix Sort memiliki kompleksitas waktu:

$$O(d(n + k))$$

yang membuat performanya sangat stabil pada data angka dengan jumlah digit yang tidak terlalu besar. Hasil pengujian membuktikan bahwa Radix Sort mampu mempertahankan waktu eksekusi paling cepat dan tidak terlalu dipengaruhi bentuk awal data.

Selain itu bentuk distribusi data juga menjadi faktor penting karena beberapa algoritma sangat sensitif terhadap kondisi awal data, sedangkan algoritma lain mampu mempertahankan performa yang lebih stabil.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-4-sorting-Fiivaldy.git>